Machine Learning (2024) 113:2351–2403
https://doi.org/10.1007/s10994-023-06495-7

Training data influence analysis and estimation: a survey

Zayd Hammoudeh[1,2] · **Daniel Lowd**[1]

Received: 13 December 2022 / Revised: 18 November 2023 / Accepted: 29 November 2023 /
Published online: 29 March 2024
© The Author(s) 2024

Abstract
Good models require good training data. For overparameterized deep models, the causal relationship between training data and model predictions is increasingly opaque and poorly understood. Influence analysis partially demystifies training's underlying interactions by quantifying the amount each training instance alters the final model. Measuring the training data's influence exactly can be provably hard in the worst case; this has led to the development and use of influence estimators, which only approximate the true influence. This paper provides the first comprehensive survey of training data influence analysis and estimation. We begin by formalizing the various, and in places orthogonal, definitions of training data influence. We then organize state-of-the-art influence analysis methods into a taxonomy; we describe each of these methods in detail and compare their underlying assumptions, asymptotic complexities, and overall strengths and weaknesses. Finally, we propose future research directions to make influence analysis more useful in practice as well as more theoretically and empirically sound.

Keywords Influence analysis · Influence estimation · Training data attribution · Data valuation · Influence functions · TracIn · Shapley value

1 Introduction

Machine learning is built on training data (Reddi et al., 2021). Without good training data, nothing else works. How modern models learn from and use training data is increasingly opaque (Koh & Liang, 2017; Zhang et al., 2021a; Xiang, 2022). Regarding state-of-the-art black-box models, Yampolskiy (2020) notes, "If all we have is a 'black box' it is impossible to understand causes of failure and improve system safety."

Large modern models require tremendous amounts of training data (Black & Fredrikson, 2021). Today's uncurated, internet-derived datasets commonly contain numerous

Editor: Jan N. van Rijn.

✉ Zayd Hammoudeh
zayd@cs.uoregon.edu

1 University of Oregon, Eugene, OR, USA

2 Qualtrics AI, Seattle, WA, USA

anomalous instances (Pleiss et al., 2020). These anomalies can arise from multiple potential sources. For example, training data anomalies may have a natural cause such as distribution shift (Rousseeuw & Leroy, 1987; Yang et al., 2021), measurement error, or non-representative samples drawn from the tail of the data distribution (Huber, 1981; Feldman, 2020b; Jiang et al., 2021). Anomalous training instances also occur due to human or algorithmic labeling errors—even on well-known, highly-curated datasets (Ekambaram et al., 2017). Malicious adversaries can insert anomalous poison instances into the training data with the goal of manipulating specific model predictions (Biggio et al., 2012; Chen et al., 2017; Shafahi et al., 2018; Hammoudeh & Lowd, 2023). Regardless of the cause, anomalous training instances degrade a model's overall generalization performance.

Today's large datasets also generally overrepresent established and dominant viewpoints (Bender et al., 2021). Models trained on these huge public datasets encode and exhibit biases based on protected characteristics, including gender, race, religion, and disability (Basta et al., 2019; Kurita et al., 2019; Tan & Celis, 2019; Zhang et al., 2020; Hutchinson et al., 2020). These training data biases can translate into real-world harm, where, as an example, a recidivism model falsely flagged black defendants as high risk at twice the rate of white defendants (Angwin et al., 2016).

Understanding the data and its relationship to trained models is essential for building trustworthy ML systems. However, it can be very difficult to answer even basic questions about the relationship between training data and model predictions; for example:

1. Is a prediction well-supported by the training data, or was the prediction just random?
2. Which portions of the training data improve a prediction? Which portions make it worse?
3. Which instances in the training set caused the model to make a specific prediction?

One strategy to address basic questions like those above is to render them moot by exclusively using simple, transparent model classes (Lipton, 2018). Evidence exists that this "interpretable-only" strategy may be appropriate in some settings (Knight, 2017). However, even interpretable model classes can be grossly affected by training data issues (Huber, 1981; Cook et al., 1982; Cook & Weisberg, 1982). Moreover, as the performance penalty of interpretable models grows, their continued use becomes harder to justify.

With the growing use of black-box models, we need better methods to analyze and understand black-box model decisions. Otherwise, society must carry the burden of black-box failures.

1.1 Relating models and their training data

All model decisions are rooted in the training data. Training data *influence analysis* [also known as *data valuation* (Ghorbani & Zou, 2019; Jia et al., 2019a; Ki et al., 2023) and *data attribution* (Park et al., 2023; Nguyen et al., 2023; Dai & Gifford, 2023)] partially demystifies the relationship between training data and model predictions by determining how to apportion credit (and blame) for specific model behavior to the training instances (Shapley and Roth, 1988; Koh & Liang, 2017; Yeh et al., 2018; Pruthi et al., 2020). Essentially, influence analysis's objective is to answer the question: *What is each training instance's effect on a model*? An instance's "effect" is with respect to some specific perspective. For example, an instance's effect may be quantified as the change in model performance

when some instance is deleted from the training data. The effect can also be relative, e.g., whether one training instance changes the model more than another.

Influence analysis emerged alongside the initial study of linear models and regression (Jaeckel, 1972; Cook & Weisberg, 1982). This early analysis focused on quantifying how worst-case perturbations to the training data affected the final model parameters. The insights gained from early influence analysis contributed to the development of numerous methods that improved model robustness and reduced model sensitivity to training outliers (Hogg, 1979; Rousseeuw, 1994).

Since these early days, machine learning models have grown substantially in complexity and opacity (Devlin et al., 2019; Krizhevsky et al., 2012; Dosovitskiy et al., 2021). Training datasets have also exploded in size (Black & Fredrikson, 2021). These factors combine to make training data influence analysis significantly more challenging where, for multilayer parametric models (e.g., neural networks), determining a single training instance's exact effect can be NP-complete in the worst case (Blum & Rivest, 1992).

In practice, influence may not need to be measured exactly. *Influence estimation* methods provide an approximation of training instances' true influence. Influence estimation is generally much more computationally efficient and is now the approach of choice (Schioppa et al., 2022). However, modern influence estimators achieve their efficiency via various assumptions about the model's architecture and learning environment (Koh & Liang, 2017; Yeh et al., 2018; Ghorbani & Zou, 2019). These varied assumptions result in influence estimators having different advantages and disadvantages as well as in some cases, even orthogonal perspectives on the definition of influence itself (Pruthi et al., 2020).

1.2 Our contributions

To the extent of our knowledge, there has not yet been a comprehensive review of these differing perspectives of training data influence, much less of the various methods themselves. This paper fills in that gap by providing the first comprehensive survey of existing influence analysis techniques. We describe how these various methods overlap and, more importantly, the consequences—both positive and negative—that arise out of their differences. We provide this broad and nuanced understanding of influence analysis so that ML researchers and practitioners can better decide which influence analysis method best suits their specific application objectives (Schioppa et al., 2022).

Although we aim to provide a comprehensive survey of influence analysis, we cannot cover every method in detail. Instead, we focus on the most impactful methods so as not to distract from the key takeaways. In particular, we concentrate on influence analysis methods that are general (Cook & Weisberg, 1982; Ghorbani & Zou, 2019; Feldman & Zhang, 2020) or targeted towards parametric models (Koh & Liang, 2017; Yeh et al., 2018; Pruthi et al., 2020; Chen et al., 2021) with less emphasis on non-parametric methods (Sharchilev et al., 2018; Jia et al., 2019a; Brophy et al., 2023). Multiple other research areas are based on ranking and subsampling training instances including data pruning (Yang et al., 2023), coreset selection (Bachem et al., 2017; Feldman, 2020a), active learning (Ren et al., 2021), and submodular dataset selection (Wei et al., 2015), but these topics are beyond the scope of this work. [1]

[1] Sect. 3.3 briefly contrasts how the objectives of these related areas align with influence analysis's objectives.

In the remainder of this paper, we first standardize the general notation used throughout this work (Sect. 2). Section 3 reviews the various general formulations through which training data influence is viewed. We also categorize and summarize the properties of the seven most impactful influence analysis methods. Sections 4 and 5 describe these foundational influence methods in detail. For each method, we (1) formalize the associated definition of influence and how it is measured, (2) detail the formulation's strengths and weaknesses, (3) enumerate any related or derivative methods, and (4) explain the method's time, space, and storage complexities. Section 6 reviews various learning tasks where influence analysis has been applied. We provide our perspective on future directions for influence analysis research in Sect. 7.

2 General notation

This section details our primary notation. In cases where a single influence method requires custom nomenclature, we introduce the unique notation alongside discussion of that method.[2] Supplemental Section A provides a full nomenclature reference; see Tables 2, 3, and 4.

Let $[r]$ denote the set of integers $\{1, \ldots, r\}$. $A \overset{m}{\sim} B$ denotes that the cardinality of set A is m and that A is drawn uniformly at random (u.a.r.) from set B. For *singleton* set A (i.e., $|A| = 1$), the sampling notation is simplified to $A \sim B$. Let 2^A denote the *power set* of any set A. Set subtraction is denoted $A \backslash B$. For singleton $B = \{b\}$, set subtraction is simplified to $A \setminus b$.

The *zero vector* is denoted $\mathbf{0}$ with the vector's dimension implicit from context. $\mathbb{1}[a]$ is the *indicator function*, where $\mathbb{1}[a] = 1$ if predicate a is true and 0 otherwise.

Let $x \in \mathcal{X} \subseteq \mathbb{R}^d$ denote an arbitrary *feature vector*, and let $y \in \mathcal{Y}$ be a *dependent value* (e.g., label, target). *Training set*, $\mathcal{D} := \{z_i\}_{i=1}^n$, consists of n training instances where each instance is a tuple, $z_i := (x_i, y_i) \in \mathcal{Z}$ and $\mathcal{Z} := \mathcal{X} \times \mathcal{Y}$. (Arbitrary) test instances are denoted $z_{\text{te}} := (x_{\text{te}}, y_{\text{te}}) \in \mathcal{Z}$. Note that y_{te} need not be x_{te}'s true dependent value; y_{te} can be any value in $\mathcal{Y}$. Throughout this work, subscripts "i" and "te" entail that the corresponding symbol applies to an arbitrary training and test instance, respectively.

Model $f : \mathcal{X} \to \mathcal{Y}$ is parameterized by $\theta \in \mathbb{R}^p$, where $p := |\theta|$; f is trained on (a subset of) dataset $\mathcal{D}$. Most commonly, f performs either classification or regression, although more advanced model classes (e.g., generative models) are also considered. Model performance is evaluated using a *loss function* $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$. Let $\mathcal{L}(z;\theta) := \ell\big(f(x;\theta), y\big)$ denote the *empirical risk* of instance $z = (x, y)$ w.r.t. parameters θ. By convention, a smaller risk is better.

This work primarily focuses on overparameterized models with $p \gg d$, where d is the data dimension. Such models are almost exclusively trained using first-order optimization algorithms (e.g., gradient descent), which proceed iteratively over T iterations. Starting from *initial parameters* $\theta^{(0)}$, the optimizer returns at the end of each iteration, $t \in [T]$, updated model parameters $\theta^{(t)}$, where $\theta^{(t)}$ is generated from previous parameters $\theta^{(t-1)}$, loss function ℓ, *batch* $\mathcal{B}^{(t)} \subseteq \mathcal{D}$, *learning rate* $\eta^{(t)} > 0$, and *weight decay* (L_2) strength $\lambda \geq 0$. Training gradients are denoted $\nabla_\theta \mathcal{L}(z_i;\theta^{(t)})$. The training set's *empirical risk Hessian* for iteration t is denoted $H_\theta^{(t)} := \frac{1}{n} \sum_{z_i \in \mathcal{D}} \nabla_\theta^2 \mathcal{L}(z_i;\theta^{(t)})$, with the corresponding *inverse risk Hessian* denoted $(H_\theta^{(t)})^{-1}$. Throughout this work, superscript "(t)" entails that the corresponding symbol applies to training iteration t.

[2] Supplemental Table 4 provides a reference for all notation specific to a single influence analysis method.

Some models may be trained on data other than full training set $\mathcal{D}$, e.g., subset $\mathcal{D} \setminus z_i$. Let $D \subset \mathcal{D}$ denote an alternate training set, and denote model parameters trained on D as $\theta_D^{(t)}$. For example, $\theta_{\mathcal{D}\setminus z_i}^{(T)}$ are the final parameters for a model trained on all of $\mathcal{D}$ except training instance $z_i \in \mathcal{D}$. When training on all of the training data, subscript $\mathcal{D}$ is dropped, i.e., $\theta^{(t)} \equiv \theta_{\mathcal{D}}^{(t)}$.

3 Overview of training data influence and influence estimation

As Sect. 1 explains, training data influence's objective is to quantify the "effect" of one or more training instances on a model. This effect's scope can be as localized as an individual model prediction, e.g., $f(x_{\text{te}};\theta^{(T)})$; the effect's scope can also be so broad as to encompass the entire test data distribution.

Positive influence entails that the training instance(s) improve some quality measure, e.g., risk $\mathcal{L}(z_{\text{te}};\theta^{(T)})$. Negative influence means that the training instance(s) make the quality measure worse. Training instances with positive influence are referred to as *proponents* or *excitatory examples*. Training instances with negative influence are called *opponents* or *inhibitory examples* (Koh & Liang, 2017; Yeh et al., 2018).

Highly expressive, overparameterized models remain functionally black boxes (Koh & Liang, 2017). Understanding why a model behaves in a specific way remains a significant challenge (Belle & Papantonis, 2021), and the inclusion or removal of even a single training instance can drastically change a trained model's behavior (Rousseeuw, 1994; Black & Fredrikson, 2021). In the worst case, quantifying one training instance's influence may require repeating all of training.

Since measuring influence *exactly* may be intractable or unnecessary, *influence estimators*—which only approximate the true influence—are commonly used in practice. As with any approximation, influence estimation requires making trade-offs, and the various influence estimators balance these design choices differently. This in turn leads influence estimators to make different assumptions and rely on different mathematical formulations.

When determining which influence analysis methods to highlight in this work, we relied on two primary criteria: (1) a method's overall impact and (2) the method's degree of novelty in relation to other approaches. In particular, we concentrate on influence analysis methods that are either model architecture agnostic or that are targeted towards parametric models (e.g., neural networks). Nonetheless, we briefly discuss non-parametric methods as well.

The remainder of this section considers progressively more general definitions of influence.

3.1 Pointwise training data influence

Pointwise influence is the simplest and most commonly studied definition of influence. It quantifies how a single training instance affects a model's prediction on a single test instance according to some quality measure (e.g., test loss). Throughout this work, we denote the pointwise influence of training instance z_i on test instance z_{te} as $\mathcal{I}(z_i, z_{\text{te}})$. Formally, a pointwise influence analysis method is a function $\mathcal{I} : \mathcal{Z} \times \mathcal{Z} \to \mathbb{R}$ with the pointwise influence of training instance z_i on test instance z_{te} denoted $\mathcal{I}(z_i, z_{\text{te}})$. Pointwise influence estimates are denoted $\widehat{\mathcal{I}}(z_i, z_{\text{te}}) \in \mathbb{R}$. Note that the model architecture, training algorithm, full training set ($\mathcal{D}$), and even the random seed can (significantly) affect an

instance's influence. To improve clarity and readability, we treat these parameters as fixed and implicit in our nomenclature for $\mathcal{I}$ and $\widehat{\mathcal{I}}$.

Below, we briefly review early pointwise influence analysis contributions and then transition to a discussion of more recent pointwise methods.

3.1.1 Early pointwise influence analysis

The earliest notions of pointwise influence emerged out of robust statistics—specifically the analysis of training outliers' effects on linear regression models (Cook, 1977). Given training set $\mathcal{D}$, the least-mean squares linear model parameters are[3]

$$\theta^* := \arg\min_\theta \frac{1}{|\mathcal{D}|} \sum_{(x_i,y_i)\in\mathcal{D}} \left(y_i - \theta^\intercal x_i\right)^2. \tag{1}$$

Observe that this least-squares estimator has a *breakdown point* of 0 (Rousseeuw, 1994). This means that least-squares regression is completely non-robust where a single training data outlier can shift model parameters θ^* arbitrarily. For example, Fig. 1 visualizes how a single training data outlier (●) can induce a nearly orthogonal least-squares model. Put simply, an outlier training instance's potential pointwise influence on a least-squares model is unbounded. Unbounded influence on the model parameters equates to unbounded influence on model predictions.

Early influence analysis methods sought to identify the training instance that was most likely to be an outlier (Srikantan, 1961; Tietjen et al., 1973). A training outlier can be defined as the training instance with the largest negative influence on prediction $f(x_{\text{te}};\theta^*)$.

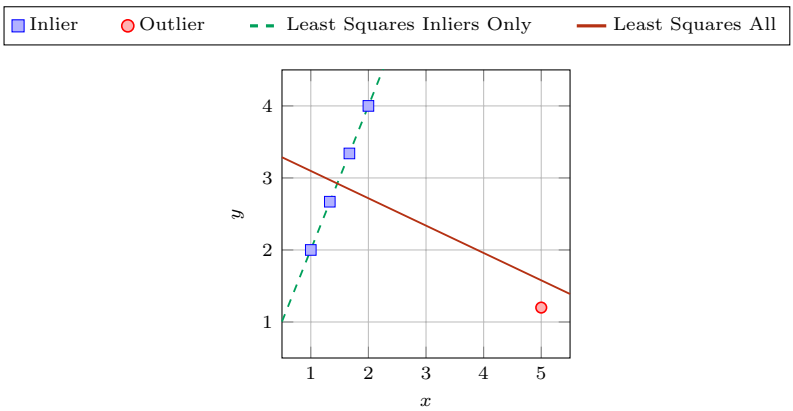

Fig. 1 Outlier pointwise influence on least-squares regression: influence of a single outlier (●) on a least-squares model where in-distribution data (■) are generated from linear distribution $y = 2x$. The single outlier sample ($x = 5$ & $y = 1.2$) influences the inlier-only least-squares linear model (- - -) substantially such that a least-squares model trained on all instances (——) predicts all training y values poorly. Adapted from Rousseeuw & Leroy (1987, Fig. 2b)

[3] For simplicity, the bias term is considered part of θ.

Intuitively, each training instance's pointwise influence can be measured by training $n := |\mathcal{D}|$ models, where each model's training set leaves out a different training instance. These n models would then be compared to identify the outlier.[4] However, such repeated retraining is expensive and so more efficient pointwise influence analysis methods were studied.

Different assumptions about the training data distribution lead to different definitions of the most likely outlier. For example, Srikantan (1961), Snedecor & Cochran (1968), and Ellenberg (1976) all assume that training data outliers arise from mean shifts in normally distributed training data. Under this constraint, their methods all identify the maximum likelihood outlier as the training instance with the largest absolute *residual*, $|y_i - \theta^{*\intercal} x_i|$. However, Cook & Weisberg (1982) prove that under different distributional assumptions (e.g., a variance shift instead of a mean shift), the maximum likelihood outlier may not have the largest residual.

These early influence analysis results demonstrating least-squares fragility spurred development of more robust regressors. For instance, Rousseeuw (1994) replaces Eq. (1)'s mean operation with median; this simple change increases the breakdown point of model parameters θ^* to the maximum value, 50%. In addition, multiple robust loss functions have been proposed that constrain or cap outliers' pointwise influence (Huber, 1964; Beaton & Tukey, 1974; Jr. & Welsch, 1978; Leclerc, 1989).

As more complex models grew in prevalence, influence analysis methods similarly grew in complexity. In recent years, numerous influence analysis methods targeting deep models have been proposed. We briefly review the most impactful, modern pointwise influence analysis methods next.

3.1.2 Modern pointwise influence analysis

Figure 2 provides a taxonomy of the seven most impactful modern pointwise influence analysis methods. Below each method appears a list of closely related and derivative approaches. Modern influence analysis methods broadly categorize into two primary classes, namely:

- *Retraining-Based Methods*: Measure the training data's influence by repeatedly retraining model f using different subsets of training set $\mathcal{D}$.
- *Gradient-Based Influence Estimators*: Estimate influence via the alignment of training and test instance gradients either throughout or at the end of training.

An in-depth comparison of these influence analysis methods requires detailed analysis so we defer the extensive discussion of these two categories to Sects. 4 and 5, respectively. Table 1 summarizes the key properties of Fig. 2's seven methods—including comparing each method's assumptions (if any), strengths/weaknesses, and asymptotic complexities. These three criteria are also discussed when detailing each of these methods in the later sections.

[4] Sect. 4.1 formalizes how to measure pointwise influence by repeatedly retraining with a different training instance left out of the training set each time.

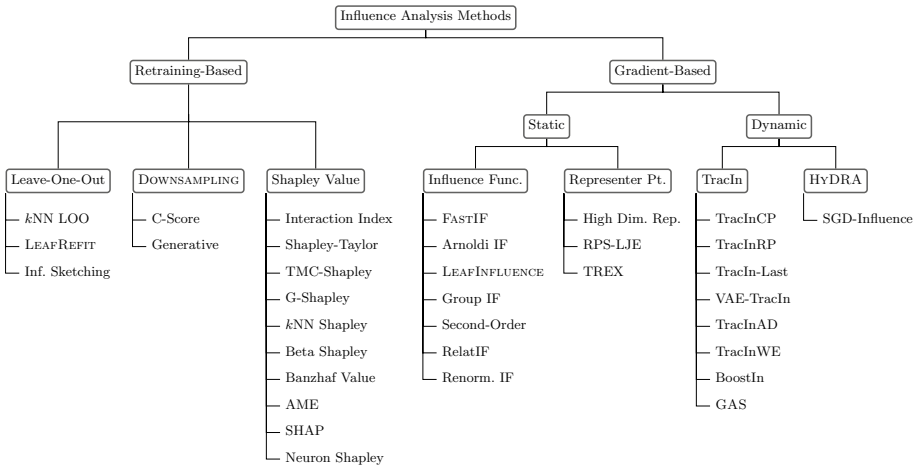

Fig. 2 Influence Analysis Taxonomy: Categorization of the seven primary pointwise influence analysis methods. Section 4 details the three primary retraining-based influence methods, leave-one-out (Sect. 4.1), Downsampling (Sect. 4.2), and Shapley value (Sect. 4.3). Section 5 details gradient-based static estimators influence functions (Sect. 5.1.1) and representer point (Sect. 5.1.2) as well as dynamic estimators TracIn (Sect. 5.2.1) and HyDRA (Sect. 5.2.2). Closely-related and derivative estimators are shown as a list below their parent method. See supplemental Table 5 for the formal mathematical definition of all influence methods and estimators. Due to space, each method's citation is in supplemental Table 6

Table 1 Influence analysis method comparison: comparison of the complexity, assumptions, strengths, and limitations of the seven primary influence estimators detailed in Sects. 4 and 5. Recall that n, p, and T are the training-set size, model parameter count, and training iteration count, respectively. LOO is the leave-one-out influence. "Full time complexity" denotes the time required to calculate influence for the first test instance; "incremental complexity" is the added time required for each subsequent test instance. "Storage complexity" represents the amount of memory required to persistently save any additional model parameters needed by the influence method, with asterisk denoting storage complexity is implementation dependent. All criteria below are discussed in detail alongside the description of each method

	Retraining-Based			Gradient-Based			
				Static		Dynamic	
	LOO	Downsamp.	Shapley	Inf. Func.	Rep. Pt.	TracIn	HyDRA
Section reference	4.1	4.2	4.3	5.1.1	5.1.2	5.2.1	5.2.2
Full time complexity	$\mathcal{O}(nT)$	$\mathcal{O}(KT)$	$\mathcal{O}(2^n\,T)$	$\mathcal{O}(np)$	$\mathcal{O}(n)$	$\mathcal{O}(npT)$	$\mathcal{O}(npT)$
Incremental complexity	$\mathcal{O}(n)$	$\mathcal{O}(K)$	$\mathcal{O}(2^n)$	$\mathcal{O}(np)$	$\mathcal{O}(n)$	$\mathcal{O}(npT)$	$\mathcal{O}(np)$
Space complexity	$\mathcal{O}(n+p)$	$\mathcal{O}(n+p)$	$\mathcal{O}(n+p)$	$\mathcal{O}(n+p)$	$\mathcal{O}(n+p)$	$\mathcal{O}(n+p)$	$\mathcal{O}(np)$
Storage complexity	$\mathcal{O}(np)^*$	$\mathcal{O}(Kp)^*$	$\mathcal{O}(2^n\,p)^*$	0	0	$\mathcal{O}(pT)$	$\mathcal{O}(pT+np)^*$
Assumes differentiable?				✓	✓*	✓	✓
Assumes convexity?				✓	✓		
Assumes stationarity?				✓	✓		
Assumes linear model?					✓		
Optimizer specific?						✓	✓
Uses Hessian?				✓			✓
Any model class?	✓	✓	✓				
Estimates LOO?	✓	✓	✓	✓			✓
Group influence?		✓	✓	✓			
Hyperparam. sensitive?				✓			✓
Generative models?		✓	✓			✓	
High upfront cost?	✓	✓	✓				✓
High instance cost?			✓	✓		✓	
Amortizable?	✓	✓	✓			✓	✓

3.2 Alternative perspectives on influence

Note that pointwise effects are only one perspective on how to analyze the training data's influence. Below we briefly summarize six alternate, albeit less common, perspectives of training data influence. While pointwise influence is this work's primary focus, later sections also contextualize existing influence methods w.r.t. these alternate perspectives where applicable.

(1) Recall that pointwise influence quantifies the effect of a single training instance on a single test prediction. In reality, multiple related training instances generally influence a prediction as a group (Feldman & Zhang, 2020), where group members have a total effect much larger than the sum of their individual effects (Basu et al., 2020; Das et al., 2021; Hammoudeh & Lowd, 2022). *Group influence* quantifies a set of training instances' total, combined influence on a specific test prediction.

We use very similar notation to denote group and pointwise influence. The only difference is that for group influence, the first parameter of function $\mathcal{I}$ is a training (sub)set instead of an individual training instance; the same applies to group influence estimates $\widehat{\mathcal{I}}$. For example, given some test instance z_{te}, the entire training set's group influence and group influence estimate are denoted $\mathcal{I}(\mathcal{D}, z_{\text{te}})$ and $\widehat{\mathcal{I}}(\mathcal{D}, z_{\text{te}})$, respectively.

In terms of magnitude, previous work has shown that the group influence of a related set of training instances is generally lowered bounded by the sum of the set's pointwise influences (Koh et al., 2019). Put simply, the true influence of a coherent group is *more than the sum of its parts*, or formally, for coherent $D \subseteq \mathcal{D}$, it often holds that

$$|\mathcal{I}(D, z_{\text{te}})| > \sum_{z_i \in D} |\mathcal{I}(z_i, z_{\text{te}})|. \tag{2}$$

Existing work studying group influence is limited. Later sections note examples where any of the seven primary pointwise influence methods have been extended to consider group effects.

(2) *Joint influence* extends influence to consider *multiple test instances* collectively (Jia et al., 2022; Chen et al., 2022). These test instances may be a specific subpopulation within the test distribution—for example in *targeted data poisoning attacks* (Jagielski et al., 2021; Wallace et al., 2021). The test instances could also be a representative subset of the entire test data distribution—for example in *coreset selection* (Borsos et al., 2020) or *indiscriminate poisoning attacks* (Biggio et al., 2012; Fowl et al., 2021).

Most (pointwise) influence analysis methods are *additive* meaning for target set $\mathcal{D}_{\text{te}} \subseteq \mathcal{Z}$, the joint (pointwise) influence simplifies to

$$\mathcal{I}(z_i, \mathcal{D}_{\text{te}}) = \sum_{z_{\text{te}} \in \mathcal{D}_{\text{te}}} \mathcal{I}(z_i, z_{\text{te}}). \tag{3}$$

Additivity is not a requirement of influence analysis, and there are provably non-additive influence estimators (Yan & Procaccia, 2021).

(3) Overparameterized models like deep networks are capable of achieving near-zero training loss in most settings (Bartlett et al., 2020; Feldman, 2020b; D'Amour et al., 2020). This holds even if the training set is large and randomly labeled (Zhang et al., 2017; Arpit

et al., 2017). Near-zero training loss occurs because deep models often memorize some training instances.

Both Pruthi et al. (2020) and Feldman & Zhang (2020) separately define a model's *memorization*[5] of training instance z_i as the pointwise influence of z_i *on itself*. Formally

$$\textsc{Mem}(z_i) := \mathcal{I}(z_i, z_i) \approx \widehat{\mathcal{I}}(z_i, z_i). \tag{4}$$

(4) *Cook's distance* measures the effect of training instances on the model parameters themselves (Cook, 1977, Eq. (5)). Formally, the pointwise Cook's distance of $z_i \in \mathcal{D}$ is

$$\mathcal{I}_{\text{Cook}}(z_i) := \theta^{(T)} - \theta^{(T)}_{\mathcal{D}\setminus z_i}. \tag{5}$$

Eq. (5) trivially extends to groups of training instances where for any $D \subseteq \mathcal{D}$

$$\mathcal{I}_{\text{Cook}}(D) := \theta^{(T)} - \theta^{(T)}_{\mathcal{D}\setminus D}. \tag{6}$$

Cook's distance is particularly relevant for interpretable model classes where feature weights are most transparent. This includes linear regression (Rousseeuw & Leroy, 1987; Wojnowicz et al., 2016) and decision trees (Brophy et al., 2023).

(5) All definitions of influence above consider training instances' effects w.r.t. a single instantiation of a model. Across repeated stochastic retrainings, a training instance's influence may vary—potentially substantially (Basu et al., 2021; Summers & Dinneen, 2021; Raste et al., 2022). *Expected influence* is the average influence across all possible instantiations within a given model class (K & Søgaard, 2021; Wang & Jia, 2023). Expected influence is particularly useful in domains where the random component of training is unknowable a priori. For example, with poisoning and backdoor attacks, an adversary crafts malicious training instances to be highly influential in expectation across all random parameter initializations and batch orderings (Chen et al., 2017; Shafahi et al., 2018; Fowl et al., 2021).

Expected influence generalizes to consider group effects. Existing related work focuses on counterfactuals such as, "what is the expected prediction for x_{te} if a model is trained on some arbitrary subset of $\mathcal{D}$ (Ilyas et al., 2022; Ki et al., 2023)?" Other work seeks to predict model parameters $\theta^{(T)}$ given an arbitrary training subset (Zeng et al., 2023).

(6) Observe that all preceding definitions view influence as a specific numerical value to measure/estimate. Influence analysis often simplifies to a relative question of whether one training instance is more influential than another. An *influence ranking* orders (groups of) training instances from most positively influential to most negatively influential. These rankings are useful in a wide range of applications (Kwon & Zou, 2022; Wang & Jia, 2023), including data cleaning and poisoning attack defenses as discussed in Sect. 6.

3.3 Topics related to influence analysis

All influence analysis methods we highlight in Sects. 4 and 5 estimate a function that quantifies the impact of specific training examples on a given model or prediction. For the most part, these methods take an ablation perspective, i.e., measuring how much a model changes

[5] Pruthi et al. (2020) term "memorization" as *self-influence*. We use Feldman & Zhang's (2020) terminology here since it is more consistent with other work (van den Burg & Williams, 2021; Kandpal et al., 2022).

when removing specific training examples. However, there are several other research areas related to analyzing the impact of different subsets of the training data, albeit with somewhat different methods or objectives. We briefly describe some of these topics below.[6]

Data pruning methods such as coresets (Bachem et al., 2017) also consider the impact of removing examples, but they typically consider removing many examples (rather than one or a few) with the primary goal of increasing computational efficiency. A *coreset* $D_{\text{CS}} \subset \mathcal{D}$ is a (weighted) set of points that can stand in for the overall training data when measuring a *cost function*, $\text{cost}(D_{\text{CS}}, Q)$ where $Q \in \mathcal{Q}$ is a solution in solution space $\mathcal{Q}$. D_{CS} is an ε *-coreset* if it approximates the cost function within a factor of $\epsilon > 0$:

$$|\text{cost}(D_{\text{CS}}, Q) - \text{cost}(\mathcal{D}, Q)| \leq \varepsilon \, \text{cost}(\mathcal{D}, Q). \tag{7}$$

Several techniques exist to efficiently find D_{CS} (Mirzasoleiman et al., 2020; Feldman, 2020a; Tukan et al., 2023). These include methods loosely based on weighted *importance sampling*, where each training instance's sampling probability is proportional to the instance's influence (Bachem et al., 2017).

Coresets let us learn a model over a much smaller set of points, while still yielding a model that is within a factor of $1 + \varepsilon$ of the optimal loss on the original training set. Therefore, a coreset represents a *sufficient* set of points for a given task, while most influence estimation methods identify the most *necessary* points — the points without which performance would decline, even given many other points from the original dataset. Another difference is that coresets are often motivated by efficiency concerns, whereas influence estimation is more motivated by the need to understand the data and its impact on a model—specifically, a model trained on the entire training data.

Coreset construction often involves *submodular optimization* (Bilmes, 2022), so that an efficient, greedy approach finds a nearly-optimal set of points. However, this also means that if there are multiple, equally-important points, submodular optimization will select one and skip the others as redundant. This "winner-take-all" approach is in stark contrast to most influence estimation methods, which tend to assign similar importance to similar points.

Active learning seeks to *maximize* a model's performance while annotating as little training data as possible (Ren et al., 2021). Like influence estimation, active learning estimates the relative value different data points would have in fitting a model. However, unlike influence estimation, this is done without knowledge of the labels of these points. Furthermore, the goal is maximizing performance more than understanding the data, increasing efficiency, or precisely matching the loss on the full training data.

Often, the data points to label are chosen greedily by identifying the training instance whose labeling would most positively influence the model (in expectation). Quantifying each unlabeled instance's true influence at each active learning iteration may be prohibitive, so influence *estimation* techniques are often used to quantify each remaining unlabeled instance's marginal contribution at a given iteration (Liu et al., 2021).

With this broad perspective on influence analysis and related concepts in mind, we transition to focusing on specific influence analysis methods in the next two sections.

[6] Sect. 6 ("Applications of Influence Analysis") discusses additional tasks that have leveraged influence analysis to achieve their own meta-objectives (e.g., adversarial robustness, model explainability, etc.).

4 Retraining-based influence analysis

Training instances can only influence a model if they are used during training. As Sect. 3.1.2 describes in the context of linear regression, one method to measure influence just trains a model with and without some instance; influence is then defined as the difference in these two models' behavior. This basic intuition is the foundation of retraining-based influence analysis, and this simple formulation applies to any model class—parametric or non-parametric.

Observe that the retraining-based framework makes no assumptions about the learning environment. In fact, this simplicity is one of the primary advantages of retraining-based influence. For comparison, Table 1 shows that all gradient-based influence estimators make strong assumptions—some of which are known not to hold for deep models (e.g., convexity). However, retraining's flexibility comes at the expense of high (sometimes prohibitive) computational cost.

Below, we describe three progressively more complex retraining-based influence analysis methods. Each method mitigates weaknesses of the preceding method—in particular, devising techniques to make retraining-based influence more viable computationally.

Remark 1 This section treats model training as *deterministic* where, given a fixed training set, training always yields the same output model. Since the training of modern models is mostly stochastic, retraining-based estimators should be represented as expectations over different random initializations and batch orderings. Therefore, (re)training should be repeated multiple times for each relevant training (sub)set with a probabilistic average taken over the valuation metric (Lin et al., 2022). For simplicity of presentation, expectation over randomness is dropped from the influence and influence estimator definitions below.

Remark 2 Sect. 2 defines $\mathcal{D}$ as a supervised training set. The three primary retraining-based influence analysis methods detailed below also generalize to unsupervised and semi-supervised training.

Remark 3 When calculating retraining's time complexity below, each training iteration's time complexity is treated as a constant cost. This makes the time complexity of training a single model $\mathcal{O}(T)$. Depending on the model architecture and hyperparameter settings, a training iteration's complexity may directly depend on training-set size n or model parameter count p.

Remark 4 It may be possible to avoid full model retraining by using machine unlearning methods capable of certifiably "forgetting" training instances (Guo et al., 2020; Brophy & Lowd, 2021; Nguyen et al., 2022; Eisenhofer et al., 2022). The asymptotic complexity of such methods is model-class specific and beyond the scope of this work. Nonetheless, certified deletion methods can drastically reduce the overhead of retraining-based influence analysis.

4.1 Leave-one-out influence

Leave-one-out (LOO) is the simplest influence measure described in this work. LOO is also the oldest, dating back to Cook & Weisberg (1982)

who term it *case deletion diagnostics*.

As its name indicates, *leave-one-out influence* is the change in z_{te}'s risk due to the removal of a single instance, z_i, from the training set (Koh & Liang, 2017; Black & Fredrikson, 2021; Jia et al., 2021). Formally,

$$\mathcal{I}_{\text{LOO}}\left(z_i, z_{\text{te}}\right) := \mathcal{L}(z_{\text{te}};\theta^{(T)}_{\mathcal{D}\setminus z_i}) - \mathcal{L}(z_{\text{te}};\theta^{(T)}), \tag{8}$$

where $\theta^{(T)}_{\mathcal{D}\setminus z_i}$ are the final model parameters when training on subset $\mathcal{D}\setminus z_i$ and $\theta^{(T)}$ are the final model parameters trained on all of $\mathcal{D}$.

Measuring the entire training set's LOO influence requires training $(n+1)$ models. Given a deterministic model class and training algorithm [e.g., convex model optimization (Boyd & Vandenberghe, 2004)], LOO is one of the few influence measures that can be computed exactly in polynomial time w.r.t. training-set size n and iteration count T.

4.1.1 Time, space, and storage complexity

Training a single model has time complexity $\mathcal{O}(T)$ (see Remark 3). By additivity, training $(n+1)$ models has total time complexity $\mathcal{O}(nT)$. Since these $(n+1)$ models are independent, they can be trained in parallel.

Pointwise influence analysis always has space complexity of at least $\mathcal{O}(n)$, i.e., the space taken by the n influence values $\forall_i\, \mathcal{I}\left(z_i, z_{\text{te}}\right)$. Training a single model has space complexity $\mathcal{O}(p)$, where $p := |\theta|$; this complexity scales linearly with the number of models trained in parallel. Table 1 treats the level of training concurrency as a constant factor, which is why LOO's total space complexity is listed as $\mathcal{O}(n+p)$.

A naive implementation of LOO would train the n additional models and immediately discard them after measuring z_{te}'s test loss. This simple version of LOO has $\mathcal{O}(1)$ storage complexity. If instead the $(n+1)$ models are stored, analysis of subsequent test instances requires no additional retraining. This drastically reduces LOO's incremental time complexity for subsequent instances to just $\mathcal{O}(n)$ forward passes—a huge saving.[7] Note that this amortization of the retraining cost induces an $\mathcal{O}(np)$ storage complexity as listed in Table 1.

4.1.2 Strengths and weaknesses

Leave-one-out influence's biggest strength is its simplicity. LOO is human-intelligible—even by laypersons. For that reason, LOO has been applied to ensure the fairness of algorithmic decisions (Black & Fredrikson, 2021). Moreover, like all methods in this section, LOO's simplicity allows it to be combined with any model architecture.

LOO's theoretical simplicity comes at the price of huge upfront computational cost. Training some state-of-the-art models from scratch even once is prohibitive for anyone beyond industrial actors (Diskin et al., 2021). For even the biggest players, it is impractical to train $(n+1)$ such models given huge modern datasets (Basu et al., 2021). The climate effects of such retraining also cannot be ignored (Strubell et al., 2020).

LOO's simple definition in Eq. (8) is premised on deterministic training. However, even when training on the *same* data, modern models may have significant predictive variance for a given test instance (Black & Fredrikson, 2021; Wang & Jia, 2023). This variance makes it difficult to disentangle the effect of an instance's deletion from training's intrinsic

[7] LOO's incremental computational cost can be (significantly) reduced in practice via batching.

variability (Basu et al., 2021). For a single training instance, estimating the LOO influence within a standard deviation of σ requires training $\Omega(1/\sigma^2)$ models. Therefore, estimating the entire training set's LOO influence requires training $\Omega(n/\sigma^2)$ models—further exacerbating LOO's computational infeasibility (Feldman & Zhang, 2020).

These limitations notwithstanding, LOO's impact on influence analysis research is substantial. Many pointwise influence analysis methods either directly estimate the leave-one-out influence—e.g., Downsampling (Sec. 4.2), influence functions (Sec. 5.1.1), HyDRA (Sec. 5.2.2)—or are very similar to LOO—e.g., Shapley value (Sec. 4.3).

4.1.3 Related methods

Efficient nearest-neighbor LOO Although LOO has poor upfront and storage complexities in general, it can be quite efficient for some model classes—particularly instance-based learners (Aha et al., 1991). For example, Jia et al. (2021) propose the kNN *leave-out-one* (kNN LOO) estimator, which calculates the LOO influence over a surrogate k-nearest neighbors classifier instead of over target model f. kNN LOO relies on a simple two-step process. First, the features of test instance z_{te} and training set $\mathcal{D}$ are extracted using a pretrained model. Next, a kNN classifier's LOO influence is calculated exactly (Jia et al., 2021, Lemma 1) using these extracted features. Jia et al. prove that kNN LOO influence only requires $\mathcal{O}(n \log n)$ time—significantly faster in practice than vanilla LOO's $\mathcal{O}(nT)$ complexity. Jia et al. also demonstrate empirically that kNN LOO and vanilla LOO generate similar influence rankings across various learning domains and tasks.

Efficient LOO estimation in decision tree ensembles Sharchilev et al. (2018) propose LeafRefit, an efficient LOO estimator for decision-tree ensembles. LeafRefit's efficiency derives from the simplifying assumption that instance deletions do not affect the trees' structure. In cases where this assumption holds, LeafRefit's tree influence estimates are exact. To the extent of our knowledge, LeafRefit's suitability for surrogate influence analysis of deep models has not yet been explored.

Cook's distance and linear regression For least-squares linear regression, Wojnowicz et al. (2016) show that each training instance's LOO influence on the model parameters (i.e., Cook's distance) can be efficiently estimated by mapping training set $\mathcal{D}$ into a lower-dimensional subspace. By the Johnson-Lindenstrauss lemma (Johnson & Lindenstrauss, 1984), these influence sketches approximately preserve the pairwise distances between the training instances in $\mathcal{D}$ provided the projected dimension is on the order of $\log n$.

LOO group influence Leave-one-out can be extended to *leave-m -out* for any integer $m \leq n$.[8] Leave-m-out has time complexity $\mathcal{O}\left(\binom{n}{m}\right)$, which is exponential in the worst case. Shapley value influence (Shapley, 1953) (Sec. 4.3) shares significant similarity with leave-m-out.

As mentioned above, LOO influence serves as the reference influence value for multiple influence estimators including Downsampling, which we describe next.

[8] Leave-m-out influence analysis is also called *multiple case deletion diagnostics* (Rousseeuw and Leroy, 1987).

4.2 Downsampling

Proposed by Feldman & Zhang (2020), Downsampling[9] mitigates leave-one-out influence's two primary weaknesses: (1) computational complexity dependent on n and (2) instability due to stochastic training variation.

Downsampling relies on an ensemble of K submodels each trained on a u.a.r. subset of full training set $\mathcal{D}$. Let $D^k \overset{m}{\sim} \mathcal{D}$ be the k-th submodel's training set where $\forall_k\, |D^k| = m < n$.[10] Define $K_i := \sum_{k=1}^{K} \mathbb{1}[z_i \in D^k]$ as the number of submodels that used instance z_i during training. The Downsampling *pointwise influence estimator*[11] is then

$$\widehat{\mathcal{I}}_{\text{Down}}\left(z_i, z_{\text{te}}\right) := \frac{1}{K - K_i} \sum_{\substack{k \\ z_i \notin D^k}} \mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D^k}) - \frac{1}{K_i} \sum_{\substack{k' \\ z_i \in D^{k'}}} \mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D^{k'}}). \tag{9}$$

Intuitively, Eq. (9) is the change in z_{te}'s average risk when z_i is not used in submodel training. By holding out multiple instances simultaneously and then averaging, each Downsampling submodel provides insight into the influence of *all* training instances. This allows Downsampling to require (far) fewer retrainings than LOO.

Since each of the K training subsets is i.i.d., then

$$\lim_{K\to\infty} \widehat{\mathcal{I}}_{\text{Down}}\left(z_i, z_{\text{te}}\right) = \mathbb{E}_{D' \overset{m}{\sim} \mathcal{D}^{\setminus z_i}}[\mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D'})] - \mathbb{E}_{D \overset{m-1}{\sim} \mathcal{D}^{\setminus z_i}}[\mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D \cup z_i})]. \tag{10}$$

For sufficiently large m and n, the expected behavior of a model trained on an i.i.d. dataset of size m becomes indistinguishable from one trained on $m - 1$ i.i.d. instances. Applying this property along with linearity of expectation and Eqs. (8), (10) reformulates as

$$\lim_{K,n,m\to\infty} \widehat{\mathcal{I}}_{\text{Down}}\left(z_i, z_{\text{te}}\right) = \mathbb{E}_{D' \overset{m-1}{\sim} \mathcal{D}^{\setminus z_i}}[\mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D'})] - \mathbb{E}_{D \overset{m-1}{\sim} \mathcal{D}^{\setminus z_i}}[\mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D \cup z_i})] \tag{11}$$

$$= \mathbb{E}_{D \overset{m-1}{\sim} \mathcal{D}^{\setminus z_i}}[\mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D}) - \mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D \cup z_i})] \tag{12}$$

$$= \mathbb{E}_{D \overset{m-1}{\sim} \mathcal{D}^{\setminus z_i}}[\mathcal{I}_{\text{LOO}}\left(z_i, z_{\text{te}}\right)]. \tag{13}$$

Hence, Downsampling is a statistically *consistent* estimator of the *expected* LOO *influence*. This means that Downsampling does not estimate the influence of training instance z_i on a single model instantiation. Rather, Downsampling estimates z_i's *influence on the*

[9] Feldman & Zhang (2020) do not specify a name for their influence estimator. Previous work has referred to (Feldman and Zhang 2020)'s method as "subsampling" (Brophy et al., 2023) and as "counterfactual influence" (Zhang et al., 2021b). We use "Downsampling" to differentiate Feldman and Zhang (2020) method from the existing, distinct task of dataset subsampling (Ting & Brochu, 2018) while still emphasizing the methods' reliance on repeated training-set sampling.

[10] Feldman & Zhang (2020) propose setting $m = \lceil 0.7n \rceil$.

[11] Feldman & Zhang (2020) define their estimator specifically for classification. Downsampling's definition in Eq. (9) uses a more general form to cover additional learning tasks such as regression. Feldman & Zhang (2020) original formulation would be equivalent to defining the risk as $\mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D^k}) = \mathbb{1}\left[y_{\text{te}} \neq f(x_{\text{te}}; \theta^{(T)}_{D^k})\right]$, i.e., the accuracy subtracted from one.

training algorithm and model architecture as a whole. By considering influence in expectation, Downsampling addresses LOO's inaccuracy caused by stochastic training's implicit variance.

In practice, K, n, and m are finite. Nonetheless, Feldman & Zhang (2020, Lemma 2.1) prove that, with high probability, Downsampling's LOO influence estimation error is bounded given K and $\frac{m}{n}$.

While Downsampling's formulation above is w.r.t. a single training instance, Downsampling trivially extends to estimate the *expected group influence* of multiple training instances. Observe however that the expected fraction of u.a.r. training subsets that either contains all instances in a group or none of a group decays geometrically with $\frac{m}{n}$ and $(1 - \frac{m}{n})$, respectively. Therefore, for large group sizes, K needs to be exponential in n to cover sufficient group combinations.

Remark 5 Downsampling trains models on data subsets of size m under the assumption that statements made about those models generalize to models trained on a dataset of size n (i.e., all of $\mathcal{D}$). This assumption may not hold for small m. To increase the likelihood this assumption holds, Feldman & Zhang (2020) propose fixing $\frac{m}{n} = 0.7$. This choice balances satisfying the aforementioned assumption against the number of submodels since Downsampling requires K and ratio $\frac{m}{n}$ combined dictate K_i, i.e., the number of submodels that are trained on z_i.

4.2.1 Time, space, and storage complexity

Downsampling's complexity analysis is identical to that of LOO (Sec. 4.1.1) except, instead of the time and storage complexities being dependent on training-set size n, Downsampling depends on submodel count K. For perspective, Feldman & Zhang (2020) empirical evaluation used $K = 2,000$ for ImageNet (Deng et al., 2009) ($n > 14$M) as well as $K = 4,000$ for MNIST (LeCun et al., 1998) ($n = 60,000$) and CIFAR10 (Krizhevsky et al., 2014) ($n = 50,000$)—a savings of one to four orders of magnitude over vanilla LOO.

Remark 6 Downsampling's incremental time complexity is technically $\mathcal{O}(K + n) \in \mathcal{O}(n)$ since pointwise influence is calculated w.r.t. each training instance. The time complexity analysis above focuses on the difference in the number of forward passes required by LOO and Downsampling; fewer forward passes translate to Downsampling being much faster than LOO in practice.

4.2.2 Strengths and weaknesses

Although more complicated than LOO, Downsampling is still comparatively simple to understand and implement. Downsampling makes only a single assumption that should generally hold in practice (see Remark 5). Downsampling is also flexible and can be applied to most applications.

Another strength of Downsampling is its low incremental time complexity. Each test example requires only K forward passes. These forward passes can use large batch sizes

to further reduce the per-instance cost. This low incremental cost allows DOWNSAMPLING to be applied at much larger scales than other methods. For example, Feldman & Zhang (2020) measure all pointwise influence estimates for the entire ImageNet dataset (Deng et al., 2009) ($n > 14$M). These large-scale experiments enabled Feldman & Zhang (2020) to draw novel conclusions about neural training dynamics—including that training instance memorization (4) by overparameterized models is not a bug, but a feature, that is currently necessary to achieve state-of-the-art generalization results.

In terms of weaknesses, while DOWNSAMPLING is less computationally expensive than LOO, DOWNSAMPLING still has a high upfront computational cost. Training multiple models may be prohibitively expensive even when $K \ll n$. Amortization of this upfront training overhead across multiple test instances is beneficial but by no means a panacea.

4.2.3 Related methods

DOWNSAMPLING has two primary related methods.

Generative DOWNSAMPLING Training instance memorization also occurs in generative models where the generated outputs are (nearly) identical copies of training instances (Kandpal et al., 2022). van den Burg & Williams (2021) extend DOWNSAMPLING to deep generative models—specifically, density models [e.g., variational autoencoders (Kingma & Welling, 2014; Rezende et al., 2014)] that estimate posterior probability $p(x|\mathcal{P}, \theta)$, where $\mathcal{P}$ denotes the training data distribution. Like DOWNSAMPLING, van den Burg and Williams (2021) approach relies on training multiple submodels.[12] The primary difference is that van den Burg and Williams (2021) consider generative risk

$$\mathcal{L}(x_{\text{te}};\theta^{(T)}) = -\log p(x_{\text{te}}|\mathcal{P}, \theta^{(T)}). \tag{14}$$

Beyond that, van den Burg and Williams (2021) method is the same as DOWNSAMPLING as both methods consider the LOO influence (9).

Consistency profile and score DOWNSAMPLING's second closely related method is Jiang et al. (2021) *consistency profile*, defined formally as,[13]

$$C_{m,\mathcal{D}}(z_{\text{te}}) := -\mathbb{E}_{D \overset{m}{\sim} \mathcal{D}}[\mathcal{L}(z_{\text{te}};\theta_D^{(T)})]. \tag{15}$$

By negating the risk in Eq. (15), a higher expected risk corresponds to a lower consistency profile. Consistency profile differs from DOWNSAMPLING in two ways. (1) DOWNSAMPLING implicitly considers a single submodel training-set size m while consistency profile disentangles the estimator from m. (2) DOWNSAMPLING estimates z_i's influence on z_{te} while consistency profile considers all of $\mathcal{D}$ as a group and estimates the expected group influence of a random subset $D \subseteq \mathcal{D}$ given $m := |D|$.

Jiang et al. (2021) also propose the *consistency score* (C-score), defined formally as

$$C_{\mathcal{D}}(z_{\text{te}}) := \mathbb{E}_{m \sim [n]}[C_{m,\mathcal{D}}(z_{\text{te}})], \tag{16}$$

[12] Rather than training submodels using i.i.d. subsets of $\mathcal{D}$, van den Burg & Williams (2021) propose training the submodels via repeated d-fold cross-validation. While technically different, van den Burg and Williams's approach is functionally equivalent to Feldman and Zhang's (2020) u.a.r. sampling procedure.

[13] Jiang et al. (2021) define their estimator specifically for classification. We present their method more generally to apply to other losses/domains (e.g., regression). As with DOWNSAMPLING it is trivial to map between Eq. (15)'s formulation and that of (Jiang et al. 2021).

where m is drawn uniformly from set $[n]$. By taking the expectation over all training-set sizes, C-score provides a total ordering over all test instances. A large C-score entails that z_{te} is harder for the model to confidently predict. Large C-scores generally correspond to rare/atypical test instances from the tails of the data distribution. Since Downsampling considers the effect of each training instance individually, Downsampling may be unable to identify these hard-to-predict test instances—in particular if m is large enough to cover most data distribution modes.

The next section introduces the Shapley value, which in essence merges the ideas of Downsampling and C-score.

4.3 Shapley value

Derived from cooperative game theory, *Shapley value* (SV) quantifies the increase in value when a group of players cooperates to achieve some shared objective (Shapley, 1953; Shapley & Roth, 1988). Given n total players, *characteristic function* $v : 2^{[n]} \to \mathbb{R}$ defines the value of any player coalition $A \subseteq [n]$ (Dubey, 1975). By convention, a larger $v(A)$ is better. Formally, player i's Shapley value w.r.t. v is

$$\mathcal{V}(i;v) := \frac{1}{n} \sum_{A \subseteq [n] \setminus i} \frac{1}{\binom{n-1}{|A|}} [v(A \cup i) - v(A)], \tag{17}$$

where $\binom{n-1}{|A|}$ is the binomial coefficient.

Ghorbani & Zou (2019) adapt SV to model training by treating the n instances in training set $\mathcal{D}$ as n cooperative players with the shared objective of training the "best" model.[14] For any $A \subseteq \mathcal{D}$, let $v(A) := -\mathcal{L}(z_{\text{te}};\theta_A^{(T)})$ where the negation is needed because more "valuable" training subsets have lower risk. Then, z_i's *Shapley value pointwise influence* on z_{te} is

$$\mathcal{I}_{\text{SV}}(z_i, z_{\text{te}}) := \frac{1}{n} \sum_{D \subseteq \mathcal{D}^{\setminus z_i}} \frac{1}{\binom{n-1}{|D|}} [\mathcal{L}(z_{\text{te}};\theta_D^{(T)}) - \mathcal{L}(z_{\text{te}};\theta_{D \cup z_i}^{(T)})]. \tag{18}$$

More intuitively, SV is the weighted change in z_{te}'s risk when z_i is added to a random training subset; the weighting ensures all training subset sizes ($|D|$) are prioritized equally. Equation (18) can be viewed as generalizing the leave-one-out influence, where rather than considering only full training set $\mathcal{D}$, Shapley value averages the LOO influence across all possible subsets of $\mathcal{D}$.

There exists multiple extensions of Shapley value to the group context (Bordt & von Luxburg, 2023; Tsai et al., 2023a; Grabisch & Roubens, 1999; Sundararajan et al., 2020).[15] The most well-known method is Grabisch and Roubens's (1999) *Shapley interaction index*, which for any subset $A \subseteq \mathcal{D}$, is defined as

[14] "Best" here is w.r.t. some data valuation measure of interest, with different use cases potentially defining "best" differently.

[15] Most of these works were proposed in the context of studying the interaction between groups of features. Directly adapting these ideas to groups of training instances is straightforward.

$$\mathcal{I}_{\mathrm{SV}}\big(A, z_{\mathrm{te}}\big) := -\sum_{D\subseteq\mathcal{D}\setminus A} \frac{(n-|A|-|D|)!\ |D|!}{(n-|A|+1)!} \sum_{D'\subseteq A} (-1)^{|A|-|D'|}\mathcal{L}(z_{\mathrm{te}};\theta^{(T)}_{D\cup D'}). \tag{19}$$

To intuitively understand Shapley interaction index's inner summation, consider when A consists of only two instances (e.g., $A = \{z_i, z_j\}$), then

$$\begin{aligned}\sum_{D'\subseteq\{z_i,z_j\}} (-1)^{2-|D'|}\mathcal{L}(z_{\mathrm{te}};\theta^{(T)}_{D\cup D'}) =& \mathcal{L}(z_{\mathrm{te}};\theta^{(T)}_{D}) - \mathcal{L}(z_{\mathrm{te}};\theta^{(T)}_{D\cup z_i}) \\ &- \mathcal{L}(z_{\mathrm{te}};\theta^{(T)}_{D\cup z_j}) + \mathcal{L}(z_{\mathrm{te}};\theta^{(T)}_{D\cup\{z_i,z_j\}}).\end{aligned} \tag{20}$$

Grabisch & Roubens (1999) explain that a positive interaction index entails that the combined group influence of the training instances in $A \subseteq \mathcal{D}$ exceeds the sum of their marginal influences; similarly, a negative interaction index means that A's group influence is less than the sum of their marginal influences. If the interaction index is zero, A's members have no net interaction. Sundararajan et al. (2020) propose an alternate Shapley group formulation they term the *Shapley-Taylor interaction index*, which is defined as

$$\mathcal{I}_{\mathrm{ST}}\big(A, z_{\mathrm{te}}\big) := -\frac{|A|}{n} \sum_{D\subseteq\mathcal{D}\setminus A} \frac{1}{\binom{n-1}{|D|}} \sum_{D'\subseteq A} (-1)^{|A|-|D'|}\mathcal{L}(z_{\mathrm{te}};\theta^{(T)}_{D\cup D'}). \tag{21}$$

The Shapley and Shapley-Taylor interaction indices provide different mathematical guarantees the details of which extend beyond the scope of this work. We refer the reader to Sundararajan et al. (2020) for additional discussion.

4.3.1 Time, space, and storage complexity

Deng & Papadimitriou (1994) prove that computing Shapley values is #P-complete. Therefore, in the worst case, SV requires exponential time to determine exactly assuming P ≠ NP. There has been significant follow-on work to develop tractable SV estimators, many of which are reviewed in Sect. 4.3.3.

The analysis of SV's space, storage, and incremental time complexities follows that of LOO (Sect. 4.1.1) with the exception that SV requires up to 2^n models, not just n models as with LOO.

4.3.2 Strengths and weaknesses

Among all influence analysis methods, SV may have the strongest theoretical foundation with the chain of research extending back several decades. SV's dynamics and limitations are well understood, providing confidence in the method's quality and reliability. In addition, SV makes minimal assumptions about the nature of the cooperative game (i.e., model to be trained), meaning SV is very flexible. This simplicity and flexibility allow SV to be applied to many domains beyond dataset influence as discussed in the next section.

SV has been shown to satisfy multiple appealing mathematical axioms. First, SV satisfies the *dummy player* axiom where

$$\forall_{D \subseteq \mathcal{D} \setminus z_i} \; \mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D \cup z_i}) = \mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D}) \implies \mathcal{I}_{\text{SV}}\big(z_i, z_{\text{te}}\big) = 0. \tag{22}$$

Second, SV is *symmetrical* meaning for any $z_i, z_j \in \mathcal{D}$,

$$\forall_{D \subseteq \mathcal{D} \setminus \{z_i, z_j\}} \; \mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D \cup z_i}) = \mathcal{L}(z_{\text{te}}; \theta^{(T)}_{D \cup z_j}) \implies \mathcal{I}_{\text{SV}}\big(z_i, z_{\text{te}}\big) = \mathcal{I}_{\text{SV}}\big(z_j, z_{\text{te}}\big). \tag{23}$$

Third, SV is *linear* (Shapley, 1953; Dubey, 1975), meaning given any two data valuation metrics ν', ν'' and $\alpha', \alpha'' \in \mathbb{R}$, it holds that

$$\mathcal{V}(D; \alpha' \nu' + \alpha'' \nu'') = \alpha' \, \mathcal{V}(D; \nu') + \alpha'' \, \mathcal{V}(D; \nu''). \tag{24}$$

SV's linearity axiom makes it possible to estimate both pointwise and joint SV influences without repeating any data collection (Ghorbani & Zou, 2019). Any data value satisfying the dummy player, symmetry, and linearity axioms is referred to as a *semivalue* (Dubey et al., 1981; Kwon & Zou, 2022). Additional semivalues include leave-one-out (Sect. 4.1) and Banzhaf value (Sect. 4.3.3) (Banzhaf, 1965).

Furthermore, by evaluating training sets of different sizes, SV can detect subtle influence behavior that is missed by methods like Downsampling and LOO, which evaluate a single training-set size. Lin et al. (2022) evidence this phenomenon empirically showing that adversarial training instances (i.e., poison) can sometimes be better detected with small SV training subsets.

Concerning weaknesses, SV's computational intractability is catastrophic for non-trivial dataset sizes (Kwon & Zou, 2022). For that reason, numerous (heuristic) SV speed-ups have been proposed, with the most prominent ones detailed next.

4.3.3 Related methods

Monte Carlo Shapley Ghorbani & Zou (2019) propose two SV estimators. First, *truncated Monte Carlo Shapley* (TMC-Shapley) relies on randomized subset sampling from training set $\mathcal{D}$.[16] As a simplified description of the algorithm, TMC-Shapley relies on random permutations of $\mathcal{D}$; for simplicity, denote the permutation ordering $z_1, \ldots, z_n$. For each permutation, n models are trained where the i-th model's training set is instances $\{z_1, \ldots, z_i\}$. To measure each z_i's marginal contribution for a given permutation, TMC-Shapley compares the performance of the $(i-1)$-th and i-th models, i.e., the models trained on datasets $\{z_1, \ldots, z_{i-1}\}$ and $\{z_1, \ldots, z_i\}$, respectively. TMC-Shapley generates additional training-set permutations and trains new models until the SV estimates converge. Ghorbani and Zou (2019) state that TMC-Shapley convergence usually requires analyzing on the order of n training-set permutations. Given training each model has time complexity $\mathcal{O}(T)$ (Remark 3), TMC-Shapley's full time complexity is in general $\mathcal{O}(n^2 T)$.

Gradient Shapley TMC-Shapley may be feasible for simple models that are fast to train. For more complex systems, $\mathcal{O}(n^2)$ complete retrainings are impractical. Ghorbani and Zou (2019) also propose *Gradient Shapley* (G-Shapley), an even faster SV estimator which follows the same basic procedure as TMC-Shapley with one critical change. Rather than taking T iterations to train each model, G-Shapley assumes models are trained in just *one*

[16] The term "truncated" in TMC-Shapley refers to a speed-up heuristic used when estimating the pointwise influences of multiple training instances at the same time. Truncation is not a necessary component of the method and is not described here.

gradient step. This means that G-Shapley's full and incremental time complexity is only $\mathcal{O}(n^2)$—a substantial speed-up over TMC-Shapley's full $\mathcal{O}(n^2T)$ complexity.[17] There is no free lunch, and G-Shapley's speed-up is usually at the expense of lower influence estimation accuracy.

Efficient Nearest-Neighbor Shapley Since TMC-Shapley and G-Shapley rely on heuristics and assumptions to achieve tractability, neither method provides approximation guarantees. In contrast, Jia et al. (2019a) prove that for k-nearest neighbors classification, SV pointwise influence can be calculated exactly in $\mathcal{O}(n \lg n)$ time. Formally, SV's characteristic function for kNN classification is

$$v_{k\text{NN}}(D) := -\frac{1}{k} \sum_{y_i \in \text{Neigh}(x_{\text{te}}; D)} \mathbb{1}[y_i = y_{\text{te}}], \tag{25}$$

where $\text{Neigh}(x_{\text{te}}; D)$ is the set of k neighbors in D nearest to x_{te} and $\mathbb{1}[\cdot]$ is the indicator function. Each training instance either has no effect on Eq. (25)'s value ($z_i \notin \text{Neigh}(x_{\text{te}}; D)$ or $y_i \neq y_{\text{te}}$). Otherwise, the training instance increases the value by one ($z_i \in \text{Neigh}(x_{\text{te}} D)$ and $y_i = y_{\text{te}}$).

Assuming the training instances are sorted by increasing distance from x_{te} (i.e., x_1 is closest to x_{te} and x_n is furthest), then z_i's pointwise kNN Shapley *influence* is

$$\mathcal{I}_{k\text{NN-SV}}(z_i, z_{\text{te}}) := \frac{\mathbb{1}[y_n = y_{\text{te}}]}{n} + \sum_{j=i}^{n-1} \frac{\mathbb{1}[y_j = y_{\text{te}}] - \mathbb{1}[y_{j+1} = y_{\text{te}}]}{k} \frac{\min\{k, j\}}{j}. \tag{26}$$

Observe that $\mathcal{I}_{k\text{NN-SV}}$'s closed form is linear in n and requires no retraining at all. In fact, k NN Shapley's most computationally expensive component is sorting the training instances by distance from x_{te}. Similar to kNN LOO above, Jia et al. (2019a) propose using k NN Shapley as a surrogate SV estimator for more complex model classes. For example, kNN Shapley could be applied to the feature representations generated by a deep neural network.

Beta Shapley Recent work has also questioned the optimality of SV assigning uniform weight to each training subset size [see Eq. (17)]. Counterintuitively, Kwon and Zou (2022) show theoretically and empirically that influence estimates on larger training subsets are *more* affected by training noise than influence estimates on smaller subsets. As such, rather than assigning all data subset sizes ($|D|$) uniform weight, Kwon and Zou (2022) argue that smaller training subsets should be prioritized. Specifically, Kwon and Zou 2022 propose *Beta Shapley*, which modifies vanilla SV by weighting the training-set sizes according to a positive skew (i.e., left-leaning) beta distribution.

SV has also been applied to study other types of influence beyond training set membership. For example, *Neuron Shapley* applies SV to identify the model neurons that are most critical for a given prediction (Ghorbani & Zou, 2020). Lundberg & Lee's (2017) SHAP is a very well-known tool that applies SV to measure feature importance. For a comprehensive survey of Shapley value applications beyond training data influence, see the work of Sundararajan & Najmi (2020) and a more recent update by Rozemberczki et al. (2022).

[17] G-Shapley and TMC-Shapley have the same incremental time complexity—$\mathcal{O}(n^2)$. Both estimators' storage complexity is $\mathcal{O}(n^2p)$.

Banzhaf value Also a semivalue (Dubey et al., 1981), *Banzhaf value* (Banzhaf, 1965) is closely related to Shapley value. Formally, the Banzhaf value influence of $z_i \in \mathcal{D}$ on test instance z_{te} is

$$\mathcal{I}_{\text{Banzhaf}}\left(z_i, z_{\text{te}}\right) := \frac{1}{2^{n-1}} \sum_{D \subseteq \mathcal{D}^{\setminus z_i}} \mathcal{L}(z_{\text{te}}; \theta_D^{(T)}) - \mathcal{L}(z_{\text{te}}; \theta_{D \cup z_i}^{(T)}). \tag{27}$$

Intuitively, the primary differences between Eqs. (17) and (27) is that Shapley value assigns each subset *size* ($|D|$) equal weight while Banzhaf value assigns each *subset* (D) equal weight. Wang & Jia (2023) prove that influence rankings based on Banzhaf value are more robust to training variance than both leave-one-out and Shapley value. Wang & Jia (2023) also empirically demonstrate that Banzhaf value can (significantly) outperform SV in practice.

Like SV, Banzhaf value has exponential time complexity. Uniform, Monte Carlo sampling from power set $2^{\mathcal{D} \setminus z_i}$ provides an unbiased estimate of Eq. (27). Wang & Jia (2023) provide a more sophisticated Banzhaf value Monte Carlo sampling strategy they term *maximum sample reuse* (MSR). MSR improves the estimates' *sample complexity* by a factor of $\mathcal{O}(n)$ over uniform Monte Carlo.

5 Gradient-based influence estimation

For modern models, retraining even a few times to tune hyperparameters is very expensive. In such cases, it is prohibitive to retrain an entire model just to gain insight into a single training instance's influence.

For models trained using gradient descent, training instances only influence a model through training gradients. Intuitively then, training data influence should be measurable when the right training gradients are analyzed. This basic idea forms the basis of gradient-based influence estimation. As detailed below, gradient-based influence estimators rely on Taylor-series approximations or risk stationarity. These estimators also assume some degree of differentiability—either of just the loss function (Yeh et al., 2018) or both the model and loss (Koh & Liang, 2017; Pruthi et al., 2020; Chen et al., 2021).

The exact analytical framework each gradient-based method employs depends on the set of model parameters considered (Hammoudeh & Lowd, 2022). *Static, gradient-based methods*—discussed first—estimate the effect of retraining by studying gradients w.r.t. final model parameters $\theta^{(T)}$. Obviously, a single set of model parameters provide limited insight into the entire optimization landscape, meaning static methods generally must make stronger assumptions. In contrast, *dynamic, gradient-based influence estimators* reconstruct the training data's influence by studying model parameters throughout training, e.g., $\theta^{(0)}, \ldots, \theta^{(T)}$. Analyzing these intermediary model parameters makes dynamic methods more computationally expensive in general, but it enables dynamic methods to make fewer assumptions.

This section concludes with a discussion of a critical limitation common to all existing gradient-based influence estimators—both static and dynamic. This common weakness can cause gradient-based estimators to systematically overlook highly influential (groups of) training instances.

5.1 Static, gradient-based influence estimation

As mentioned above, static estimators are so named because they measure influence using only final model parameters $\theta^{(T)}$. Static estimators' theoretical formulations assume *stationarity* (i.e., the model parameters have converged to a risk minimizer) and *convexity*.

Below we focus on two static estimators—*influence functions* (Koh & Liang, 2017) and *representer point* (Yeh et al., 2018). Each method takes very different approaches to influence estimation with the former being more general and the latter more scalable. Both estimators' underlying assumptions are generally violated in deep networks.

5.1.1 Influence functions

Along with Shapley value (Sec. 4.3), Koh & Liang's (2017) *influence functions* is one of the best-known influence estimators. The estimator derives its name from influence functions [also known as *infinitesimal jackknife* (Jaeckel, 1972)] in robust statistics (Hampel, 1974). These early statistical analyses consider how a model changes if training instance z_i's weight is infinitesimally perturbed by ϵ_i. More formally, consider the change in the *empirical risk minimizer* from

$$\theta^{(T)} = \arg\min_\theta \frac{1}{n} \sum_{z \in \mathcal{D}} \mathcal{L}(z;\theta) \tag{28}$$

to

$$\theta^{(T)}_{+\epsilon_i} = \arg\min_\theta \frac{1}{n} \sum_{z \in \mathcal{D}} \mathcal{L}(z;\theta) + \epsilon_i \mathcal{L}(z_i;\theta). \tag{29}$$

Under the assumption that model f and loss function ℓ are twice-differentiable and strictly convex, Cook & Weisberg (1982) prove via a first-order Taylor expansion that

$$\frac{d\theta^{(T)}_{+\epsilon_i}}{d\epsilon_i}\Big|_{\epsilon_i=0} = -(H^{(T)}_\theta)^{-1} \nabla_\theta \mathcal{L}(z_i;\theta^{(T)}), \tag{30}$$

where empirical risk Hessian $H^{(T)}_\theta := \frac{1}{n} \sum_{z \in \mathcal{D}} \nabla^2_\theta \mathcal{L}(z;\theta^{(T)})$ is by assumption positive definite. Koh & Liang (2017) extend Cook and Weisberg Cook & Weisberg (1982) result to consider the effect of this infinitesimal perturbation on z_{te}'s risk, where

$$\frac{d\mathcal{L}(z_{\text{te}};\theta^{(T)})}{d\epsilon_i}\Big|_{\epsilon_i=0} = \frac{d\mathcal{L}(z_{\text{te}};\theta^{(T)})}{d\theta^{(T)}_{+\epsilon_i}}^{\intercal} \frac{d\theta^{(T)}_{+\epsilon_i}}{d\epsilon_i}\Big|_{\epsilon_i=0} \qquad \triangleright \text{ Chain rule} \tag{31}$$

$$= -\nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(T)})^{\intercal} (H^{(T)}_\theta)^{-1} \nabla_\theta \mathcal{L}(z_i;\theta^{(T)}). \tag{32}$$

Removing training instance z_i from $\mathcal{D}$ is equivalent to $\epsilon_i = -\frac{1}{n}$ making the *pointwise influence functions estimator*

$$\widehat{\mathcal{I}}_{\text{IF}}(z_i, z_{\text{te}}) := \frac{1}{n} \nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(T)})^{\intercal} (H^{(T)}_\theta)^{-1} \nabla_\theta \mathcal{L}(z_i;\theta^{(T)}). \tag{33}$$

More intuitively, Eq. (33) is the influence functions' estimate of the leave-one-out influence of z_i on z_{te}.

Time, Space, and Storage Complexity

Calculating inverse Hessian $(H_\theta^{(T)})^{-1}$ directly requires $\mathcal{O}(np^2 + p^3)$ time and $\mathcal{O}(p^2)$ space (Koh & Liang, 2017). For large models, this is clearly prohibitive. Rather than computing $(H_\theta^{(T)})^{-1}$ directly, Koh & Liang (2017) instead estimate *Hessian-vector product* (HVP)

$$s_{\text{test}} := (H_\theta^{(T)})^{-1} \nabla_\theta \mathcal{L}(z_{\text{te}}; \theta^{(T)}), \tag{34}$$

with each training instance's pointwise influence then

$$\widehat{\mathcal{I}}_{\text{IF}}\left(z_i, z_{\text{te}}\right) = \frac{1}{n} s_{\text{test}}^{\text{T}} \nabla_\theta \mathcal{L}(z_i; \theta^{(T)}). \tag{35}$$

Koh & Liang (2017) use the stochastic algorithm of Pearlmutter (1994) and Agarwal et al. (2017) to estimate s_{test}. This reduces influence functions' complexity to just $\mathcal{O}(np)$ time and $\mathcal{O}(n + p)$ space. Since s_{test} is specific to test instance z_{te}, s_{test} must be *estimated for each test instance individually*. This increases influence functions' computational cost but also means that influence functions require no additional storage.

Strengths and Weaknesses

Influence functions' clear advantage over retraining-based methods is that influence functions eliminate the need to retrain any models. Recall from Table 1 that retraining-based methods have a high upfront complexity of retraining but low incremental complexity. Influence functions invert this computational trade-off. While computing s_{test} can be slow [several hours for a single test instance (Yeh et al., 2018; Guo et al., 2021; Hammoudeh & Lowd, 2022)], it is still significantly faster than retraining $\mathcal{O}(n)$ models. However, influence functions require that s_{test} is recalculated for each test instance. Hence, when a large number of test instances need to be analyzed, *retraining-based methods may actually be faster than influence functions* through amortization of the retraining cost.

In addition, while influence functions can be very accurate on convex and some shallow models, the assumption that Hessian $H_\theta^{(T)}$ is positive definite often does not hold for deep models (Basu et al., 2021). To ensure inverse $(H_\theta^{(T)})^{-1}$ exists, Koh and Liang (2017) add a small dampening coefficient to the matrix's diagonal; this dampener is a user-specified hyperparameter that is problematic to tune, since there may not be a ground-truth reference. When this dampening hyperparameter is set too small, s_{test} estimation can diverge (Yeh et al., 2018; Hara et al., 2019; Hammoudeh & Lowd, 2022).

More generally, Basu et al. (2021) show that training hyperparameters also significantly affect influence functions' performance. Specifically, Basu et al. (2021) empirically demonstrate that model initialization, model width, model depth, weight-decay strength, and even the test instance being analyzed (z_{te}) all can negatively affect influence functions' LOO estimation accuracy. Their finding is supported by the analysis of Zhang & Zhang (2022) who show that HVP estimation's accuracy depends heavily on the model's training regularizer with HVP accuracy "pretty low under weak regularization."

Bae et al. (2022) also empirically analyze the potential sources of influence functions' fragility on deep models. Bae et al. identify five common error sources, the first three of which are the most important.[18]

- *Warm-start gap*: Influence functions more closely resembles performing fine-tuning close to final model parameters $\theta^{(T)}$ than retraining from scratch (i.e., starting from initial parameters $\theta^{(0)}$). This difference in starting conditions can have a significant effect on the LOO estimate.
- *Proximity gap*: The error introduced by the dampening term included in the HVP (s_{test}) estimation algorithm.
- *Non-convergence gap*: The error due to final model parameters $\theta^{(T)}$ not being a stationary point, i.e., $\sum_{z_i \in \mathcal{D}} \nabla_\theta \mathcal{L}(z_i;\theta^{(T)}) \neq \mathbf{0}$.
- *Linearization error*: The error induced by considering only a first-order Taylor approximation when deleting z_i and ignoring the potential effects of curvature on $\widehat{\mathcal{I}}_{\text{IF}}\left(z_i, z_{\text{te}}\right)$ (Basu et al., 2020).
- *Solver error*: General error introduced by the specific solver used to estimate s_{test}.

Rather than estimating the LOO influence, Bae et al. argue that influence functions more closely estimate a different measure they term the *proximal Bregman response function* (PBRF). Bae et al. (2021) provide the intuition that PBRF "approximates the effect of removing a data point while trying to keep predictions consistent with the ... trained model" (Bae et al., 2022). Put simply, PBRF mimics a prediction-constrained LOO influence.

Bae et al. (2021) assert that PBRF can be applied in many of the same situations where LOO is useful. Bae et al. (2021) further argue that influence functions' fragility reported by earlier works (Basu et al., 2021; Zhang & Zhang, 2022) is primarily due to those works focusing on the "wrong question" of LOO. When the "right question" is posed and influence functions are evaluated w.r.t. PBRF, influence functions give accurate answers.

Related Methods

Improving influence functions' computational scalability has been a primary focus of follow-on work. For instance, applying influence functions only to the model's final linear (classification) layer has been considered with at best mixed results (Barshan et al., 2020; Yeh et al., 2022).[19] Moreover, Guo et al.'s (2021) *fast influence functions* (FastIF) integrate multiple speed-up heuristics. First, they leverage the inherent parallelizability of Pearlmutter's (1994) HVP estimation algorithm. In addition, FastIF includes recommended hyperparameters for Pearlmutter's HVP algorithm that reduces its execution time by 50% on average.

Arnoldi-based influence functions More recently, Schioppa et al. (2022) show that influence functions can be sped-up by three to four orders of magnitude by using a different algorithm to estimate s_{test}. Specifically, Schioppa et al. (2022) use Arnoldi's (1951) famous algorithm that quickly finds the dominant (in absolute terms) eigenvalues and eigenvectors of $H_\theta^{(T)}$. These dominant eigenvectors serve as the basis when projecting all gradient vectors to a lower-dimensional subspace. Schioppa et al. (2022) evaluate their revised influence functions estimator on large transformer networks [e.g., ViT-L32 with 300 M parameters (Dosovitskiy et al., 2021)], which are orders of magnitude larger than the simple networks Koh and Liang (2017) consider (Schioppa et al., 2023).

[18] (2022, Table 1) provide a formal mathematical definition of influence functions' five error sources.

[19] Sect. 5.1.2 discusses some of the pitfalls associated with estimating influence using only a model's final (linear) layer.

Influence functions for decision trees Another approach to speed up influence functions is to specialize the estimator to model architectures with favorable computational properties. For example, Sharchilev et al.'s (2018) LeafInfluence method adapts influence functions to gradient boosted decision tree ensembles. By assuming a fixed tree structure and then focusing only on the trees' leaves, LeafInfluence's tree-based estimates are significantly faster than influence functions on deep models (Brophy et al., 2023).

Group influence functions A major strength of influence functions is that it is one of the few influence analysis methods that has been studied beyond the pointwise domain. For example, Koh et al.'s (2019) follow-on paper analyzes influence functions' empirical performance estimating group influence. In particular, Koh et al. (2019) consider coherent training data subpopulations whose removal is expected to have a large, broad effect on the model. Even under naive assumptions of (pointwise) influence additivity, Koh et al. (2019) observe that simply summing influence functions estimates tends to *underestimate* the true group influence. More formally, let $D \subseteq \mathcal{D}$ be a coherent training-set subpopulation, then

$$\sum_{z_i \in D} \left|\widehat{\mathcal{I}}_{\text{IF}}\left(z_i, z_{\text{te}}\right)\right| < \left|\mathcal{I}\left(D, z_{\text{te}}\right)\right|. \tag{36}$$

Nonetheless, influence functions' additive group estimates tend to have strong rank correlation w.r.t. subpopulations' true group influence. In addition, Basu et al. (2020) extend influence functions to directly account for subpopulation group effects by considering higher-order terms in influence functions' Taylor-series approximation.

5.1.2 Representer point methods

Unlike this section's other gradient-based estimators, Yeh et al.'s (2018) *representer point* method does not directly rely on a Taylor-based expansion. Instead, Yeh et al. build on Schölkopf et al.'s (2001) *generalized representer theorem*. The derivation below assumes that model f is linear. Yeh et al. (2018) use a simple "trick" to extend linear representer point methods to multilayer models like neural networks.

Representer-based methods rely on *kernels*, which are functions $\mathcal{K} : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ that measure the similarity between two vectors (Hofmann et al., 2008). Schölkopf et al. (2001) representer theorem proves that the optimal solution to a class of L_2 regularized functions can be reformulated as a weighted sum of the training data in kernel form. Put simply, representer methods decompose the predictions of specific model classes into the individual contributions (i.e., influence) of each training instance. This makes influence estimation a natural application of the representer theorem.

Consider regularized empirical risk minimization where optimal parameters satisfy.[20]

$$\theta^* := \arg\min_\theta \frac{1}{n} \sum_{z_i \in \mathcal{D}} \mathcal{L}(z_i;\theta) + \frac{\lambda}{2}\|\theta\|_2, \tag{37}$$

with $\lambda > 0$ the L_2 regularization strength. Note that Eq. (37) defines minimizer θ^* slightly differently than the last section (28) since the representer theorem *requires* regularization.

Empirical risk minimizers are *stationary points* meaning

[20] Eq. (37) is slightly different than Yeh et al.'s (2018) presentation. This alternate formulation requires specifying λ 2× larger. We selected this alternate presentation to provide consistency with later work (Tsai et al., 2023b; Chen et al., 2021)

$$\nabla_\theta \left(\frac{1}{n} \sum_{z_i \in \mathcal{D}} \mathcal{L}(z_i;\theta^*) + \lambda \|\theta^*\|_2 \right) = \mathbf{0}, \tag{38}$$

where $\mathbf{0}$ is the p-dimensional, zero vector. The above simplifies to

$$\frac{1}{n} \sum_{z_i \in \mathcal{D}} \frac{\partial \mathcal{L}(z_i;\theta^*)}{\partial \theta} + \lambda \theta^* = \mathbf{0} \tag{39}$$

$$\theta^* = -\frac{1}{\lambda n} \sum_{z_i \in \mathcal{D}} \frac{\partial \mathcal{L}(z_i;\theta^*)}{\partial \theta}. \tag{40}$$

For a linear model where $f(x;\theta) = \theta^\intercal x =: \hat{y}$, Eq. (40) further simplifies via the chain rule to

$$\theta^* = -\frac{1}{\lambda n} \sum_{(x_i,y_i) \in \mathcal{D}} \frac{\partial \ell(f(x_i;\theta^*), y_i)}{\partial \theta} \tag{41}$$

$$= -\frac{1}{\lambda n} \sum_{(x_i,y_i) \in \mathcal{D}} \frac{\partial \ell(\hat{y}_i, y_i)}{\partial \hat{y}} \frac{\partial \hat{y}_i}{\partial \theta} \qquad \triangleright \text{ Chain Rule} \tag{42}$$

$$= -\frac{1}{\lambda n} \sum_{(x_i,y_i) \in \mathcal{D}} \frac{\partial \ell(\hat{y}_i, y_i)}{\partial \hat{y}} x_i \qquad \triangleright\ \hat{y} = \theta^\intercal x \tag{43}$$

$$= \sum_{(x_i,y_i) \in \mathcal{D}} \alpha_i x_i, \tag{44}$$

where ℓ is any once-differentiable loss function, $\frac{\partial \ell(\hat{y}_i,y_i)}{\partial \hat{y}}$ is the gradient of *just the loss function* itself w.r.t. model output $\hat{y}$,[21] and $\alpha_i := -\frac{1}{\lambda n} \frac{\partial \ell(\hat{y}_i,y_i)}{\partial \hat{y}}$ is the i-th training instance's *representer value*. Yeh et al. (2018) provide the intuition that a larger magnitude α_i indicates that training instance z_i has larger influence on the final model parameters θ^*.

Following Schölkopf et al.'s (2001) representer theorem kernelized notation, the training set's group influence on test instance z_{te} for any linear model is

$$\mathcal{I}_{\text{RP}}(\mathcal{D}, z_{\text{te}}) = \sum_{i=1}^{n} \alpha_i x_i^\intercal x_{\text{te}} \,|_{y_{\text{te}}} = \sum_{i=1}^{n} \mathcal{K}(x_i, x_{\text{te}}, \alpha_i) \,|_{y_{\text{te}}} = \sum_{i=1}^{n} \mathcal{I}_{\text{RP}}(z_i, z_{\text{te}}), \tag{45}$$

where kernel function $\mathcal{K}(x_i, x_{\text{te}}, \alpha_i) := \alpha_i x_i^\intercal x_{\text{te}}$ returns a vector. $\mathcal{K}(x_i, x_{\text{te}}, \alpha_i) \,|_{y_{\text{te}}}$ denotes the kernel value's y_{te}-th dimension. Then, z_i's *pointwise linear representer point influence* on z_{te} is

$$\mathcal{I}_{\text{RP}}(z_i, z_{\text{te}}) = \alpha_i x_i^\intercal x_{\text{te}} \,|_{y_{\text{te}}} = \mathcal{K}(x_i, x_{\text{te}}, \alpha_i) \,|_{y_{\text{te}}}. \tag{46}$$

Extending representer point to multilayer models Often, linear models are insufficiently expressive, with multilayer models used instead. In such cases, the representer theorem above does not directly apply. To workaround this limitation, Yeh et al. (2018) rely on what they (later) term *last layer similarity* (Yeh et al., 2022).

[21] For classification, $\frac{\partial \ell(\hat{y}_i,y_i)}{\partial \hat{y}}$ is $|\mathcal{Y}|$ dimensional, i.e., its dimension equals the number of classes.

Formally, Yeh et al. (2018) partition the model parameters $\theta^{(T)} = [\ \dot{\theta}^{(T)}\ \ \ddot{\theta}^{(T)}\ ]$ into two subsets, where $\ddot{\theta}^{(T)}$ is the last linear (i.e., classification) layer's parameters and $\dot{\theta}^{(T)} := \theta^{(T)} \backslash \ddot{\theta}^{(T)}$ is all other model parameters. Since $\ddot{\theta}^{(T)}$ is simply a linear function, the representer theorem analysis above still applies to it. Yeh et al. (2018) treat the other parameters, $\dot{\theta}^{(T)}$, as a fixed feature extractor and ignore them in their influence analysis.

To use Yeh et al.'s (2018) multilayer trick, one small change to Eq. (46) is required. In multilayer models, the final (linear) layer does not operate over feature vectors x_i and x_{te} directly. Instead, the final layer only sees an intermediate feature representation. For arbitrary feature vector x, let $\mathbf{f}$ be the feature representation generated by model parameters $\dot{\theta}^{(T)}$, i.e., vector $\mathbf{f}$ is the input to the model's last linear layer given input x. Then the *representer point influence estimator* for a multilayer model is

$$\widehat{\mathcal{I}}_{\text{RP}}\left(z_i, z_{\text{te}}\right) := \alpha_i \mathbf{f}_i^{\mathsf{T}} \mathbf{f}_{\text{te}} \,|_{y_{\text{te}}} = \mathcal{K}(\mathbf{f}_i, \mathbf{f}_{\text{te}}, \alpha_i) \,|_{y_{\text{te}}}. \tag{47}$$

Time, Space, and Storage Complexity

Treating as constants feature representation dimension $|\mathbf{f}|$ and the overhead to calculate $\frac{\partial \ell(\hat{y}_i, y_i)}{\partial \hat{y}}$, estimating the entire training set's representer point influence only requires calculating n dot products. This only takes $\mathcal{O}(n)$ time and $\mathcal{O}(n+p)$ space with no additional storage requirements.

Strengths and Weaknesses

Representer point's primary advantage is its theoretical and computational simplicity. Equation (47) only considers the training and test instances' final feature representations and loss function gradient $\frac{\partial \ell(\hat{y}_i, y_i)}{\partial \hat{y}}$. Hence, the majority of representer point's computation is forward-pass only and can be sped up using batching. This translates to representer point being very fast—several orders of magnitude faster than influence functions and Sect. 5.2's dynamic estimators (Hammoudeh & Lowd, 2021).

However, representer points' simplicity comes at a cost. First, at the end of training, it is uncommon that a model's final linear layer has converged to a stationary point. Before applying their method, Yeh et al. (2018) recommend freezing all model layers except the final one (i.e., freezing $\dot{\theta}^{(T)}$) and then fine-tuning the classification layer ($\ddot{\theta}^{(T)}$) until convergence/stationarity. Without this extra fine-tuning, representer point's stationarity assumption does not hold, and poor influence estimation accuracy is expected. Beyond just complicating the training procedure itself, this extra training procedure also complicates comparison with other influence methods since it may require evaluating the approaches on different parameters.

Moreover, by focusing exclusively on the model's final linear (classification) layer, representer point methods may miss influential behavior that is clearly visible in other layers. For example, Hammoudeh & Lowd (2022) demonstrate that while some training-set attacks are clearly visible in a network's final layer, other attacks are only visible in a model's first layer—despite both attacks targeting the same model architecture and dataset. In their later paper, Yeh et al. (2022) acknowledge the disadvantages of considering only the last layer writing, "that choice critically affects the similarity component of data influence and leads to inferior results." Yeh et al. (2022) further state that the feature representations in the final layer—and by extension representer point's influence estimates—can be "too reductive."

In short, Yeh et al.'s (2018) representer point method is highly scalable and efficient but is only suitable to detect behaviors that are obvious in the model's final linear layer.

Related Methods

Given the accuracy limitations of relying on last-layer similarity, limited follow-on work has adapted representer-point methods.

Adapting representer point to decision trees Brophy et al. (2023) extend representer point methods to decision forests via their Tree-ensemble Representer Point Examples (TREX) estimator. Specifically, they use *supervised tree kernels*—which provide an encoding of a tree's learned representation structure (Davies & Ghahramani, 2014; He et al., 2014)—for similarity comparison.

Making representer point more robust In addition, Sui et al. (2021) propose *representer point selection based on a local Jacobian expansion* (RPS-LJE), which can be viewed as a generalizing Yeh et al.'s (2018) base method. Rather than relying on Schölkopf et al.'s (2001) representer theorem, Sui et al. (2021) formulation relies on a first-order Taylor expansion that estimates the difference between the final model parameters and a true stationary point. RPS-LJE still follows vanilla representer point's kernelized decomposition where $\widehat{\mathcal{I}}_{\mathrm{RP}}\left(z_i, z_{\mathrm{te}}\right) := \alpha_i \mathbf{f}_i^{\intercal} \mathbf{f}_{\mathrm{te}}$, albeit with α_i defined differently.

RPS-LJE addresses two weaknesses of Yeh et al.'s (2018) base approach. First, Sui et al.'s formulation does not presume that $\theta^{(T)}$ is a stationary point. Therefore, RPS-LJE does not require post-training fine-tuning to enforce stationarity. Second, as Sect. 5.3 discusses in detail, gradient-based estimators, including vanilla representer point, tend to mark as most influential those training instances with the largest loss values. This leads to all test instances from a given class having near identical top-k influence rankings. RPS-LJE's alternate definition of α_i is less influenced by a training instance's loss value, which enables RPS-LJE to generate more semantically meaningful influence rankings.

Extending representer point to other regularizers Yeh et al.'s (2018) representer point formulation exclusively considers L_2-regularized models. Intuitively, regularization's role is to encourage the model parameters to meet certain desired properties, which may necessitate the use of alternate regularizers. For example, L_1 regularization is often used to induce sparse minimizers.

Recently, Tsai et al. (2023b) propose *high-dimensional representers*, a novel extension of Yeh et al.'s (2018) representer theorem to additional types of regularization. Specifically, Tsai et al. consider *decomposable* regularization functions (Negahban et al., 2012). Formally, a regularization function $r : \mathbb{R}^p \to \mathbb{R}_{\geq 0}$ is decomposable w.r.t. two subspaces $\mathcal{U}, \mathcal{V} \subseteq \mathbb{R}^p$ if $\forall\, u \in \mathcal{U}$ and $\forall\, v \in \mathcal{V}$,

$$r(u + v) = r(u) + r(v). \tag{48}$$

Examples of decomposable regularizers include L_1-norm (Tibshirani, 1996) and the matrix nuclear norm (Yuan et al., 2007; Recht, 2011; Yang et al., 2017).

High-dimensional representers follow Eq. (47)'s kernelized form. Representer value $\alpha_i := -\frac{1}{\lambda n} \frac{\partial \ell(\hat{y}_i, y_i)}{\partial \hat{y}}$ still quantifies the global importance of each training instance. Moreover, the similarity between training instance x_i and test instance x_{te} is still measured via a kernel function ($\mathcal{K}$). The only difference is that the kernels are specialized for these alternate regularizers; specifically the kernels are based on the decomposable regularization function's *sub-differential* (Negahban et al., 2012).

5.2 Dynamic, gradient-based influence estimation

All preceding influence methods—static, gradient-based and retraining-based—define and estimate influence using only final model parameters, $\theta_D^{(T)}$, where $D \subseteq \mathcal{D}$. These final parameters only provide a snapshot into a training instance's possible effect. Since neural

network training is NP-complete (Blum & Rivest, 1992), it can be provably difficult to reconstruct how each training instance affected the training process.

As an intuition, an influence estimator that only considers the final model parameters is akin to only reading the ending of a book. One *might* be able to draw some big-picture insights, but the finer details of the story are most likely lost. Applying a dynamic influence estimator is like reading a book from beginning to end. By comprehending the whole influence "story," dynamic methods can observe training data relationships—both fine-grained and general—that other estimators miss.

Since test instance z_{te} may not be known before model training, in-situ influence analysis may not be possible. Instead, as shown in Alg. 1, intermediate model parameters $\Theta \subseteq \{\theta^{(0)}, \dots, \theta^{(T-1)}\}$ are stored during training for post hoc influence analysis.[22]

Below we examine two divergent approaches to dynamic influence estimation—the first defines a novel definition of influence while the second estimates leave-one-out influence with fewer assumptions than influence functions.

Algorithm 1 Dynamic influence estimation's training phase

Input: Training set $\mathcal{D}$; iteration count T; learning rates $\eta^{(1)}, \dots, \eta^{(T)}$; batch sizes $b^{(1)}, \dots, b^{(T)}$; and initial parameters $\theta^{(0)}$

Output: Final parameters $\theta^{(T)}$ and stored parameter set Θ

```
Θ ← ∅
for t ← 1 to T do
    Θ ← Θ ∪ {θ^(t−1)}                    ▷ Store intermediate params.
    B^(t) ~^(b^(t)) D
    θ^(t) ← UPDATE(η^(t), θ^(t−1), B^(t))
return θ^(T), Θ
```

5.2.1 TracIn—Tracing Gradient Descent

Fundamentally, all preceding methods define influence w.r.t. changes to the training set. Pruthi et al. (2020) take an orthogonal perspective. They treat training set $\mathcal{D}$ as fixed, and consider the *change in model parameters as a function of time*, or more precisely, the training iterations.

Vacuously, the training set's group influence on test instance z_{te} is

$$\mathcal{I}\big(\mathcal{D}, z_{\text{te}}\big) = \mathcal{L}(z_{\text{te}};\theta^{(0)}) - \mathcal{L}(z_{\text{te}};\theta^{(T)}). \tag{49}$$

In words, training set $\mathcal{D}$ causes the entire change in test loss between random initial parameters $\theta^{(0)}$ and final parameters $\theta^{(T)}$. Equation (49) decomposes by training iteration t as

$$\mathcal{I}\big(\mathcal{D}, z_{\text{te}}\big) = \sum_{t=1}^{T} \big(\mathcal{L}(z_{\text{te}};\theta^{(t-1)}) - \mathcal{L}(z_{\text{te}};\theta^{(t)})\big). \tag{50}$$

Consider training a model with vanilla stochastic gradient descent, where each training minibatch $\mathcal{B}^{(t)}$ is a single instance and gradient updates have no momentum (Rumelhart

[22] In practice, only a subset of $\{\theta^{(0)}, \dots, \theta^{(T-1)}\}$ is actually stored. Heuristics are then applied to this subset to achieve acceptable influence estimation error (Pruthi et al., 2020; Hammoudeh & Lowd, 2022).

et al., 1986). Here, each iteration t has no effect on any other iteration beyond the model parameters themselves. Combining this with singleton batches enables attribution of each parameter change to a single training instance, namely whichever instance was in $\mathcal{B}^{(t)}$. Under this regime, Pruthi et al. (2020) define the *ideal TracIn pointwise influence* as

$$\mathcal{I}_{\text{TracIn}}\left(z_i, z_{\text{te}}\right) := \sum_{\substack{t \\ z_i = \mathcal{B}^{(t)}}} \left(\mathcal{L}(z_{\text{te}};\theta^{(t-1)}) - \mathcal{L}(z_{\text{te}};\theta^{(t)})\right), \tag{51}$$

where the name "TracIn" derives from "tracing gradient descent influence." Eq. (50) under vanilla stochastic gradient descent decomposes into the sum of all pointwise influences

$$\mathcal{I}\left(\mathcal{D}, z_{\text{te}}\right) = \sum_{i=1}^{n} \left(\sum_{\substack{t \\ z_i = \mathcal{B}^{(t)}}} \mathcal{L}(z_{\text{te}};\theta^{(t-1)}) - \mathcal{L}(z_{\text{te}};\theta^{(t)}) \right) = \sum_{i=1}^{n} \mathcal{I}_{\text{TracIn}}\left(z_i, z_{\text{te}}\right). \tag{52}$$

While the ideal TracIn influence has a strong theoretical motivation, its assumption of singleton batches and vanilla stochastic gradient descent is unrealistic in practice. To achieve reasonable training times, modern models train on batches of up to hundreds of thousands or millions of instances. Training on a single instance at a time would be far too slow (You et al., 2017; Goyal et al., 2017; Brown et al., 2020).

A naive fix to Eq. (51) to support non-singleton batches assigns the same influence to all instances in the minibatch, or more formally, divide the change in loss $\mathcal{L}(z_{\text{te}};\theta^{(t-1)}) - \mathcal{L}(z_{\text{te}};\theta^{(t)})$ by batch size $|\mathcal{B}^{(t)}|$ for each $z_i \in \mathcal{B}^{(t)}$. This naive approach does not differentiate those instances in batch $\mathcal{B}^{(t)}$ that had positive influence on the prediction from those that made the prediction worse.

Instead, Pruthi et al. (2020) estimate the contribution of each training instance within a minibatch via a first-order Taylor approximation. Formally,

$$\mathcal{L}\left(z_{\text{te}};\theta^{(t)}\right) \approx \mathcal{L}\left(z_{\text{te}};\theta^{(t-1)}\right) + \nabla_\theta \mathcal{L}\left(z_{\text{te}};\theta^{(t-1)}\right)^{\intercal} \left(\theta^{(t)} - \theta^{(t-1)}\right). \tag{53}$$

Under gradient descent without momentum, the change in model parameters is directly determined by the batch instances' gradients, i.e.,

$$\theta^{(t)} - \theta^{(t-1)} = -\frac{\eta^{(t)}}{|\mathcal{B}^{(t)}|} \sum_{z_i \in \mathcal{B}^{(t)}} \nabla_\theta \mathcal{L}\left(z_i;\theta^{(t-1)}\right), \tag{54}$$

where $\eta^{(t)}$ is iteration t's learning rate.

Combining Eqs. (52)–(54), the *TracIn pointwise influence estimator* is

$$\widehat{\mathcal{I}}_{\text{TracIn}}\left(z_i, z_{\text{te}}\right) := \sum_{\substack{t \\ z_i \in \mathcal{B}^{(t)}}} \frac{\eta^{(t)}}{|\mathcal{B}^{(t)}|} \nabla_\theta \mathcal{L}(z_i;\theta^{(t-1)})^{\intercal} \nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(t-1)}), \tag{55}$$

with the complete TracIn influence estimation procedure shown in Alg. 2.

A More "Practical" TracIn Training's stochasticity can negatively affect the performance of both ideal TracIn (51) and the TracIn influence estimator (55). As an intuition, consider when the training set contains two identical copies of some instance. All preceding gradient-based methods assign those two identical instances the same influence score. However, it is unlikely that those two training instances will always appear together in the same minibatch.

Therefore, ideal TracIn almost certainly assigns these identical training instances different influence scores. These assigned scores may even be vastly different—by up to several orders of magnitude (Hammoudeh & Lowd, 2022). This is despite identical training instances always having the same *expected* TracIn influence.

Pruthi et al. (2020) recognize randomness's effect on TracIn and propose the *TracIn Checkpoint influence estimator* (TracInCP) as a "practical" alternative. Rather than retrace all of gradient descent, TracInCP considers only a subset of the training iterations (i.e., checkpoints) $\mathcal{T} \subseteq [T]$. More importantly, at each $t \in \mathcal{T}$, *all* training instances are analyzed—not just those in recent batches. Equation (56) formalizes TracInCP, with its modified influence estimation procedure shown in Alg. 3.

$$\widehat{\mathcal{I}}_{\text{TracInCP}}\left(z_i, z_{\text{te}}\right) := \sum_{t \in \mathcal{T}} \eta^{(t)} \nabla_\theta \mathcal{L}(z_i; \theta^{(t-1)})^\intercal \nabla_\theta \mathcal{L}(z_{\text{te}}; \theta^{(t-1)}) \tag{56}$$

Observe that, unlike TracIn, TracInCP assigns identical training instances the same influence estimate. Therefore, TracInCP more closely estimates expected influence than TracIn. Pruthi et al. (2020) use TracInCP over TracIn in much of their empirical evaluation. Other work has also shown that TracInCP routinely outperforms TracIn on many tasks (Hammoudeh & Lowd, 2022).

Algorithm 2 TracIn influence estimation

Input: Training param. set Θ; iteration count T; batches $\mathcal{B}^{(1)}, \ldots, \mathcal{B}^{(T)}$; learning rates $\eta^{(1)}, \ldots, \eta^{(T)}$; training instance z_i; and test example z_{te}

Output: TracIn influence estimate $\widehat{\mathcal{I}}_{\text{TracIn}}\left(z_i, z_{\text{te}}\right)$

1: $\widehat{\mathcal{I}} \leftarrow 0$
2: **for** $t \leftarrow 1$ **to** T **do**
3: **if** $z_i \in \mathcal{B}^{(t)}$ **then**
4: $\theta^{(t-1)} \leftarrow \Theta[t]$
5: $\widehat{\mathcal{I}} \leftarrow \widehat{\mathcal{I}} + \frac{\eta^{(t)}}{|\mathcal{B}^{(t)}|} \nabla_\theta \mathcal{L}(z_i;\ \theta^{(t-1)})^\intercal \nabla_\theta \mathcal{L}(z_{\text{te}};\ \theta^{(t-1)})$
6: **return** $\widehat{\mathcal{I}}$

Algorithm 3 TracInCP influence estimation

Input: Training param. set Θ; iteration subset $\mathcal{T}$; learning rates $\eta^{(1)}, \ldots, \eta^{(T)}$; training instance z_i; and test example z_{te}

Output: TracInCP influence estimate $\widehat{\mathcal{I}}_{\text{TracInCP}}\left(z_i, z_{\text{te}}\right)$

1: $\widehat{\mathcal{I}} \leftarrow 0$
2: **for each** $t \in \mathcal{T}$ **do**
3: $\theta^{(t-1)} \leftarrow \Theta[t]$
4: $\widehat{\mathcal{I}} \leftarrow \widehat{\mathcal{I}} + \eta^{(t)} \nabla_\theta \mathcal{L}(z_i;\ \theta^{(t-1)})^\intercal \nabla_\theta \mathcal{L}(z_{\text{te}};\ \theta^{(t-1)})$
5: **return** $\widehat{\mathcal{I}}$

Remark 7 Pruthi et al. (2020) empirical evaluation uses $|\mathcal{T}| \ll T$. For example, when identifying mislabeled examples using TracInCP, Pruthi et al. (2020) evaluate every 30th iteration. Furthermore, Pruthi et al. note that prioritizing the small number of checkpoints where z_{te}'s loss changes significantly generally outperforms evaluating a larger number of evenly-spaced checkpoints.

Time, Space, and Storage Complexity

Below we derive the time complexity of both versions of TracIn. We then discuss their space and storage complexities.

Consider first TracInCP's time complexity since it is simpler to derive. From Alg. 3, each checkpoint in $\mathcal{T}$ requires n, p-dimensional dot products making TracInCP's complexity $\mathcal{O}(np|\mathcal{T}|)$. For vanilla TracIn, consider Alg. 2. For each iteration $t \in [T]$, a p-dimensional dot product is performed for each instance in $\mathcal{B}^{(t)}$. Let $b := \max_t |\mathcal{B}^{(t)}|$ denote the maximum batch size, then TracIn's time complexity is $\mathcal{O}(bpT)$. In the worst case where $\forall_t \mathcal{B}^{(t)} = \mathcal{D}$ (full-batch gradient descent), TracIn time complexity is $\mathcal{O}(npT)$.

Recall that, by definition, $|\mathcal{T}| \leq T$ meaning TracInCP is *asymptotically* faster than TracIn. However, this is misleading. In practice, TracInCP is generally *slower* than TracIn as Pruthi et al. note.

Since each gradient calculation is independent, TracIn and TracInCP are fully parallelizable. Table 1 treats the level of concurrency as a constant factor, making the space complexity of both TracIn and TracInCP $\mathcal{O}(n + p)$.

Lastly, as detailed in Alg. 1, dynamic influence estimators require that intermediate model parameters Θ be saved during training for post hoc influence estimation. In the worst case, each training iteration's parameters are stored resulting in a storage complexity of $\mathcal{O}(pT)$. In practice however, TracIn only considers a small fraction of these T training parameter vectors, meaning TracIn's actual storage complexity is generally (much) lower than the worst case.

Strengths and Weaknesses

TracIn and TracInCP avoid many of the primary pitfalls associated with static, gradient-based estimators.

First, recall from Sect. 5.1.1 that Hessian-vector product s_{test} significantly increases the computational overhead and potential inaccuracy of influence functions. TracIn's theoretical simplicity avoids the need to compute any Hessian.

Second, representer point's theoretical formulation necessitated considering only a model's final linear layer, at the risk of (significantly) worse performance. TracIn has the flexibility to use only the final linear layer for scenarios where that provides sufficient accuracy[23] as well as the option to use the full model gradient when needed.

Third, by measuring influence during the training process, TracIn requires no assumptions about stationarity or convergence. In fact, TracIn can be applied to a model that is only partially trained. TracIn can also be used to study *when* during training an instance is most influential. For example, TracIn can identify whether a training instance is most influential early or late in training.

Fourth, due to how gradient-based methods estimate influence, highly influential instances can actually appear *uninfluential* at the end of training. Unlike static estimators, dynamic methods like TracIn may still be able to detect these instances. See Sect. 5.3 for more details.

In terms of weaknesses, TracIn's theoretical motivation assumes stochastic gradient descent without momentum. However, momentum and adaptive optimization [e.g.,

[23] Last layer only TracIn is also referred to as TracIn-Last. Yeh et al. (2022) evaluate TracIn-Last's effectiveness.

Adam (Kingma & Ba, 2015)] significantly accelerate model convergence (Qian, 1999; Duchi et al., 2011; Kingma & Ba, 2015). To align more closely with these sophisticated optimizers, Eq. (55) and Alg. 2 would need to change significantly. For context, Sect. 5.2.2 details another dynamic estimator, HyDRA, which incorporates support for just momentum with the resulting increase in estimator complexity substantial.

Related Methods

TracIn has been adapted by numerous derivative/heuristic variants. For example, *TracIn-Last* is identical to vanilla TracIn except gradient vectors $\nabla_\theta \mathcal{L}(z_i;\theta^{(t-1)})$ and $\nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(t-1)})$ only consider the model's final linear layer (Pruthi et al., 2020). This can make TracIn significantly faster at the risk of (significantly) worse accuracy (Yeh et al., 2022).

TracIn for language models As a counter to the disadvantages of solely considering a model's last layer,[24] TracIn's authors subsequently proposed *TracIn word embeddings* (TracInWE), which targets large language models and considers only the gradients in those models' word embedding layer (Yeh et al., 2022). Since language-model word embeddings can still be very large [e.g., BERT-Base's word embedding layer has 23 M parameters (Devlin et al., 2019)], the authors specifically use the gradients of only those tokens that appear in both training instance z_i and test instance z_{te}.

Low dimensional TracIn Pruthi et al. (2020) also propose *TracIn Random Projection* (TracInRP)—a low-memory version of TracIn that provides unbiased estimates of $\widehat{\mathcal{I}}_{\text{TracIn}}$ (i.e., an estimate of an estimate). Intuitively, TracInRP maps gradient vectors into a d-dimensional subspace ($d \ll p$) via multiplication by a $d \times p$ random matrix where each entry is sampled i.i.d. from Gaussian distribution $\mathcal{N}\left(0, \frac{1}{d}\right)$. These low-memory gradient "sketches" are used in place of the full gradient vectors in Eq. (55) (Woodruff, 2014). TracInRP is primarily targeted at applications where p is sufficiently large that storing the full training set's gradient vectors ($\forall_{t,i}\ \nabla_\theta \mathcal{L}(z_i;\theta^{(t-1)})$) is prohibitive.

TracIn for generative models TracIn has also been used outside of supervised settings. For example, Kong & Chaudhuri (2021) apply TracIn to unsupervised learning, in particular density estimation; they propose *variational autoencoder TracIn* (VAE-TracIn), which quantifies the TracIn influence in β-VAEs (Higgins et al., 2017). Moreover, Thimonier et al.'s (2022) *TracIn anomaly detector* (TracInAD) functionally estimates the distribution of influence estimates—using either TracInCP or VAE-TracIn. TracInAD then marks as anomalous any test instance in the tail of this "influence distribution".

Note also that TracIn can be applied to any iterative, gradient-based model, including those that are non-parametric. For example, Brophy et al.'s (2023) *BoostIn* adapts TracIn for gradient-boosted decision tree ensembles.

5.2.2 HyDRA—hypergradient data relevance analysis

Unlike TracIn which uses a novel definition of influence (51), Chen et al.'s (2021) *hypergradient data relevance analysis* (HyDRA) estimates the leave-one-out influence (8). HyDRA leverages the same Taylor series-based analysis as Koh & Liang's (2017) influence functions. The key difference is that HyDRA addresses a fundamental mismatch between influence functions' assumptions and deep models.

[24] See Sect. 5.1.2 for an extended discussion of last-layer similarity.

Section 5.1.1 explains that influence functions consider infinitesimally perturbing the weight of training sample z_i by ϵ_i. Recall that the change in z_{te}'s test risk w.r.t. to this infinitesimal perturbation is

$$\frac{d\mathcal{L}(z_{\text{te}};\theta^{(T)})}{d\epsilon_i} = \frac{\partial\mathcal{L}(z_{\text{te}};\theta^{(T)})}{\partial\theta^{(T)}}^{\intercal}\frac{d\theta^{(T)}}{d\epsilon_i} = \frac{\partial\mathcal{L}(z_{\text{te}};\theta^{(T)})}{\partial\theta^{(T)}}^{\intercal}\widetilde{h}_i^{(T)} \tag{57}$$

where $\widetilde{h}_i^{(T)} := \frac{d\theta_{+\epsilon_i}^{(T)}}{d\epsilon_i}$ denotes the p-dimensional *hypergradient* of training instance i at the end of training.

Koh & Liang's (2017) assumptions of differentiability and strict convexity mean that Eq. (57) has a closed form. However, deep neural models are not convex. Under non-convex gradient descent without momentum and with L_2 regularization, $\theta^{(t)} := \theta^{(t-1)} - \eta^{(t)}g^{(t-1)}$ where gradient

$$g^{(t-1)} := \nabla_\theta\mathcal{L}(\mathcal{B}^{(t)};\theta^{(t-1)}) + \lambda\theta^{(t-1)}. \tag{58}$$

The exact definition of gradient $g^{(t-1)}$ depends on the specific contents of batch $\mathcal{B}^{(t)}$ so for simplicity, we encapsulate the batch's contribution to the gradient using catch-all term $\nabla_\theta\mathcal{L}(\mathcal{B}^{(t)};\theta^{(t-1)})$.

Using Eq. (58), hypergradient $\widetilde{h}_i^{(T)}$ can be defined recursively as

$$\widetilde{h}_i^{(T)} := \frac{d\theta_{+\epsilon_i}^{(T)}}{d\epsilon_i} = \frac{d}{d\epsilon_i}\left(\theta_{+\epsilon_i}^{(T-1)} - \eta^{(t)}g^{(t-1)}\right) \tag{59}$$

$$= \widetilde{h}_i^{(T-1)} - \eta^{(T)}\frac{d}{d\epsilon_i}\left(\nabla_\theta\mathcal{L}(\mathcal{B}^{(T)};\theta_{+\epsilon_i}^{(T-1)}) + \lambda\theta_{+\epsilon_i}^{(T-1)}\right) \tag{60}$$

$$= (1-\eta^{(T)}\lambda)\widetilde{h}_i^{(T-1)} - \eta^{(t)}\frac{d}{d\epsilon_i}\nabla_\theta\mathcal{L}(\mathcal{B}^{(T)};\theta_{+\epsilon_i}^{(T-1)}) \tag{61}$$

The recursive definition of hypergradient $\widetilde{h}_i^{(T)}$ needs to be unrolled all the way back to initial parameters $\theta^{(0)}$.

The key takeaway from Eq. (61) is that training hypergradients affect the model parameters *throughout all of training*. By assuming a convex model and loss, Koh and Liang's (2017) simplified formulation ignores this very real effect. As Chen et al. (2021) observe, hypergradients often cause non-convex models to converge to a vastly different risk minimizer. By considering the hypergradients' cumulative effect, HyDRA can provide more accurate LOO estimates than influence functions on non-convex models—albeit via a significantly more complicated and computationally expensive formulation.

Unrolling gradient descent hypergradients The exact procedure to unroll HyDRA's hypergradient $\widetilde{h}_i^{(T)}$ is non-trivial. For the interested reader, supplemental Section C provides hypergradient unrolling's full derivation for vanilla gradient descent without momentum. Below, we briefly summarize Section C's important takeaways, and Section C's full derivation can be skipped with minimal loss of understanding.

At each training iteration, hypergradient unrolling requires estimating the risk Hessian of each training instance in $\mathcal{B}^{(t)}$.[25] This significantly slows down HyDRA [by a factor of about 1,000× (Chen et al., 2021)]. As a workaround, Chen et al. (2021) propose treating these risk Hessians as all zeros, proving that, under mild assumptions, the approximation error of this simplified version of HyDRA is bounded. Alg. 4 shows HyDRA's *fast approximation algorithm* without Hessians for vanilla gradient descent.[26] After calculating the final hypergradient, substituting $\widetilde{h}_i^{(T)}$ into Eq. (57) with $\epsilon_i = -\frac{1}{n}$[27] yields training instance z_i's HyDRA *pointwise influence estimator*

$$\widehat{\mathcal{I}}_{\text{HyDRA}}\left(z_i, z_{\text{te}}\right) := -\frac{1}{n}\nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(T)})^\intercal \widetilde{h}_i^{(T)}. \tag{62}$$

Algorithm 4 Fast HyDRA influence estimation for gradient descent without momentum

Input: Training parameter set Θ; final parameters $\theta^{(T)}$; training set size n; iteration count T; batches $\mathcal{B}^{(1)}, \ldots, \mathcal{B}^{(T)}$; learning rates $\eta^{(1)}, \ldots, \eta^{(T)}$; weight decay λ; training instance z_i; and test example z_{te}

Output: HyDRA influence estimate $\widehat{\mathcal{I}}_{\text{HyDRA}}\left(z_i, z_{\text{te}}\right)$

```
h̃_i^(0) ← 0⃗                                          ▷ Initialize to zero vector
for t ← 1 to T do
    if z_i ∈ B^(t) then
        θ^(t−1) ← Θ[t]
        h̃_i^(t) ← (1 − η^(t) λ) h̃_i^(t−1) − (η^(t) n / |B^(t)|) ∇_θ L(z_i; θ^(t−1))
    else
        h̃_i^(t) ← (1 − η^(t) λ) h̃_i^(t−1)
Î ← −(1/n) ∇_θ L(z_te; θ^(T))^⊺ h̃_i^(T)                ▷ Influence estimate
return Î
```

Relating HyDRA **and TracIn** When $\lambda = 0$ or weight decay's effects are ignored (as done by TracIn), HyDRA's fast approximation for vanilla gradient descent simplifies to

$$\widehat{\mathcal{I}}_{\text{HyDRA}}\left(z_i, z_{\text{te}}\right) \approx -\frac{1}{n}\nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(T)})^\intercal \sum_{\substack{t \\ z_i \in \mathcal{B}^{(t)}}} -\frac{\eta^{(t)} n}{|\mathcal{B}^{(t)}|}\nabla_\theta \mathcal{L}(z_i;\theta^{(t-1)}) \tag{63}$$

[25] For clarity, this is *not* the inverse Hessian $(H_\theta^{(T)})^{-1}$ used by influence functions.

[26] See HyDRA's original paper (Chen et al., 2021) for the fast approximation algorithm with momentum.

[27] $\epsilon_i = -\frac{1}{n}$ is equivalent to deleting instance z_i from the training set. Influence functions follow the same procedure for ϵ_i. See Eqs. (32) and (33).

$$= \sum_{\substack{t \\ z_i \in \mathcal{B}^{(t)}}} \frac{\eta^{(t)}}{|\mathcal{B}^{(t)}|} \nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(T)})^\intercal \nabla_\theta \mathcal{L}(z_i;\theta^{(t-1)}) \tag{64}$$

$$= \nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(T)})^\intercal \sum_{\substack{t \\ z_i \in \mathcal{B}^{(t)}}} \frac{\eta^{(t)}}{|\mathcal{B}^{(t)}|} \nabla_\theta \mathcal{L}(z_i;\theta^{(t-1)}). \tag{65}$$

Eq. (64) is very similar to TracIn's definition in Eq. (55), despite the two methods *estimating different definitions of influence* [LOO vs. ideal TracIn (51)]. The only difference between (64) and (55) is that HyDRA always uses final test gradient $\nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(T)})$ while TracIn uses each iteration's test gradient $\nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(t-1)})$

The *key takeaway* is that while theoretically different, HyDRA and TracIn are in practice very similar where HyDRA can be viewed as trading (incremental) speed for lower precision w.r.t. z_{te}.

Time, Space, and Storage Complexity

When unrolling the T training iterations, HyDRA's fast approximation performs a p-dimensional (hyper)gradient calculation for each of the n training instances. If HyDRA's full version with Hessian vector products is used, Agarwal et al.'s (2017) Hessian approximation algorithm estimates each HVP in $\mathcal{O}(p)$ time and space. Therefore, the fast and standard versions of HyDRA both have full time complexity $\mathcal{O}(npT)$—same as TracIn, albeit with potentially much worse constant factors.

Observe that each hypergradient $\widetilde{h}_i^{(T)}$ only needs to be computed once and can be reused for each test instance. Therefore, the fast and standard version of HyDRA have incremental time complexity of just n gradient dot products—$\mathcal{O}(np)$ complexity total. This incremental complexity is much faster than TracIn and asymptotically equivalent to influence functions. In practice though, HyDRA's incremental cost is much lower than that of influence functions.

Alg. 4 requires storing vector $\widetilde{h}_i^{(t)}$ throughout HyDRA's entire unrolling procedure, where each training instance's hypergradient takes $\mathcal{O}(p)$ space. To analyze all training instances simultaneously, HyDRA requires $\mathcal{O}(np)$ total space. In contrast, TracIn only requires $\mathcal{O}(n+p)$ space to analyze all instances simultaneously. This difference is substantial for large models and training sets. In cases where the fully-parallelized space complexity is prohibitive, each training instance's hypergradient can be analyzed separately resulting in a reduced space complexity of $\mathcal{O}(p)$ for both fast and standard HyDRA.

Like TracIn, HyDRA requires storing model parameters $\Theta \subseteq \{\theta^{(0)}, \ldots, \theta^{(T-1)}\}$ making its minimum storage complexity $\mathcal{O}(pT)$. Since hypergradients are reused for each test instance, they can be stored to eliminate the need to recalculate them; this introduces an additional storage complexity of $\mathcal{O}(np)$. This makes HyDRA's total storage complexity $\mathcal{O}(pT + np)$.

Remark 8 Storing both the training checkpoints and hypergradients is unnecessary. Once all hypergradients have been calculated, serialized training parameters Θ are no longer needed and can be discarded. Therefore, a more typical storage complexity is $\mathcal{O}(pT)$ or $\mathcal{O}(np)$—both of which are still substantial.

Strengths and Weaknesses

HyDRA and TracIn share many of the same strengths. For example, HyDRA does not require assumptions of convexity or stationarity. Moreover, as a dynamic method, HyDRA may be able to detect influential examples that are missed by static methods—in particular when those instances have low loss at the end of training (see Sect. 5.3 for more discussion).

HyDRA also has some advantages over TracIn. For example, as shown in Alg. 3, TracIn requires that each *test* instance be retraced through the entire training process. This significantly increases TracIn's incremental time complexity. In contrast, HyDRA only unrolls gradient descent for the *training* instances, i.e., not the test instances. Hypergradient unrolling is a one-time cost for each training instance; this upfront cost is amortized over all test instances. Once the hypergradients have been calculated, HyDRA is much faster than TracIn—potentially by orders of magnitude. In addition, HyDRA's overall design allows it to natively support momentum with few additional changes. Integrating momentum into TracIn, while theoretically possible, requires substantial algorithmic changes and makes TracIn substantially more complicated. This would mitigate a core strength of TracIn—its simplicity.

HyDRA does have two weaknesses in comparison to TracIn. First, HyDRA's standard (i.e., non-fast) algorithm requires calculating many HVPs. Second, HyDRA's $\mathcal{O}(np)$ space complexity is much larger than the $\mathcal{O}(n+p)$ space complexity of other influence analysis methods (see Table 1). For large models, this significantly worse space complexity may be prohibitive.

Related Methods

The method most closely related to HyDRA is Hara et al.'s (2019) *SGD-influence*. Both approaches estimate the leave-one-out influence by unrolling gradient descent using empirical risk Hessians. There are, however, a few key differences. First, unlike HyDRA, Hara et al. (2019) assume that the model and loss function are convex. Next, SGD-influence primarily applies unrolling to quantify the Cook's distance, $\theta^{(T)} - \theta^{(T)}_{\mathcal{D}\setminus z_i}$. To better align their approach with dataset influence, Hara et al. (2019) propose a surrogate (linear) influence estimator which they incrementally update throughout unrolling. This means the full training process must be unrolled for each test instance individually, significantly increasing SGD-influence's incremental time complexity.

Terashita et al. (2021) adapt the ideas of SGD-influence to estimate training data influence in generative adversarial networks (GANs).

Although proposed exclusively in the context of influence functions (Sect. 5.1.1), Schioppa et al.'s (2022) basic approach to scale up influence functions via faster Hessian calculation could similarly be applied to speed up HyDRA's standard (non-fast) algorithm.

5.3 Trade-off between gradient magnitude and direction

This section details a limitation common to existing gradient-based influence estimators that can cause these estimators to systematically overlook highly influential (groups of) training instances.

Observe that all gradient-based methods in this section rely on some vector dot product. For a dot product to be large, one of two criteria must be met:

(1) The vector directions align (i.e., have high cosine similarity). More specifically, for influence analysis, vectors pointing in similar directions are expected to encode similar information. This is the ideal case.

(2) Either vector has a large magnitude, e.g., $\|\nabla_\theta \mathcal{L}(z;\theta)\|$. Large gradient magnitudes can occur for many reasons, but the most common cause is that the instance is either incorrectly or not confidently predicted.

Across the training set, gradient magnitudes can vary by several orders of magnitude (Sui et al., 2021). To overcome such a magnitude imbalance, training instances that actually influence a specific prediction may need to have orders of magnitude better vector alignment. In reality, what commonly happens is that incorrectly predicted or abnormal training instances appear highly influential to all test instances (Sui et al., 2021). Barshan et al. (2020) describe such training instances as *globally influential*. However, globally influential training instances provide very limited insight into individual model predictions. As Barshan et al. (2020) note, *locally influential* training instances are generally much more relevant and insightful when analyzing specific predictions.

Relative influence To yield a more semantically meaningful influence ranking, Barshan et al. (2020) propose the *θ -relative influence functions estimator* (RelatIF), which normalizes Schioppa et al.'s (2017) influence functions' estimator by HVP magnitude $\|(H_\theta^{(T)})^{-1}\,\nabla_\theta \mathcal{L}(z_i;\theta^{(T)})\|$.[28] Formally,

$$\widehat{\mathcal{I}}_{\text{RelatIF}}\big(z_i, z_{\text{te}}\big) := \frac{\widehat{\mathcal{I}}_{\text{IF}}\big(z_i, z_{\text{te}}\big)}{\|(H_\theta^{(T)})^{-1}\,\nabla_\theta \mathcal{L}(z_i;\theta^{(T)})\|} = \frac{1}{n}\,\frac{\nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(T)})^\intercal (H_\theta^{(T)})^{-1} \nabla_\theta \mathcal{L}(z_i;\theta^{(T)})}{\|(H_\theta^{(T)})^{-1}\,\nabla_\theta \mathcal{L}(z_i;\theta^{(T)})\|}. \tag{66}$$

RelatIF's normalization inhibits training gradient magnitude $\|\nabla_\theta \mathcal{L}(z_i;\theta^{(T)})\|$ dominating the influence estimate.

RelatIF's biggest limitation is the need to estimate an HVP *for every training instance*. As discussed in Sect. 5.1.1, HVP estimation is expensive and often highly inaccurate in deep models. To work around these issues in their evaluation of RelatIF, Barshan et al. (2020) use either very small neural models or just consider a large model's final layer, both of which can be problematic.

Renormalized influence Hammoudeh & Lowd (2022) make a similar observation as Barshan et al. (2020) but motivate it differently. By the chain rule, gradient vectors decompose as

$$\nabla_\theta \mathcal{L}(z;\theta) := \frac{\partial \ell(f(x;\theta), y)}{\partial \theta} = \frac{\partial \ell(f(x;\theta), y)}{\partial f(x;\theta)}\,\frac{\partial f(x;\theta)}{\partial \theta}. \tag{67}$$

Hammoudeh & Lowd (2022) note that for many common loss functions (e.g., squared, binary cross-entropy), loss value $\ell(f(x;\theta), y)$ induces a strict ordering over loss norm $\|\frac{\partial \ell(f(x;\theta),y)}{\partial f(x;\theta)}\|$. Hammoudeh & Lowd (2022) term this phenomenon a *low-loss penalty*, where confidently predicted training instances have smaller gradient magnitudes and by consequence consistently appear uninfluential to gradient-based influence estimators.

To account for the low-loss penalty, Hammoudeh & Lowd (2022) propose *renormalized influence* which replaces all gradient vectors—both training and test—in an influence estimator with the corresponding *unit vector*. Renormalization can be applied to any

[28] Note that this HVP is different than $s_{\text{test}} := (H_\theta^{(T)})^{-1} \nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(T)})$ in Eq. (34).

gradient-based estimator. For example, Hammoudeh and Lowd (2022) observe that renormalized TracInCP, which they term *gradient aggregated similarity* (GAS),

$$\widehat{\mathcal{I}}_{\text{GAS}}\left(z_i, z_{\text{te}}\right) := \sum_{t \in \mathcal{T}} \eta^{(t)} \frac{\nabla_\theta \mathcal{L}(z_i;\theta^{(t-1)})^\intercal \nabla_\theta \mathcal{L}(z_{te};\theta^{(t-1)})}{\left\|\nabla_\theta \mathcal{L}(z_i;\theta^{(t-1)})\right\| \left\|\nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(t-1)})\right\|}, \tag{68}$$

is particularly effective at generating influence rankings. Hammoudeh and Lowd (2022) also provide a renormalized version of influence functions,

$$\widehat{\mathcal{I}}_{\text{RenormIF}}\left(z_i, z_{\text{te}}\right) := \frac{\widehat{\mathcal{I}}_{\text{IF}}\left(z_i, z_{\text{te}}\right)}{\|\nabla_\theta \mathcal{L}(z_i;\theta^{(T)})\|} = \frac{1}{n} \frac{\nabla_\theta \mathcal{L}(z_{\text{te}};\theta^{(T)})^\intercal (H_\theta^{(T)})^{-1} \nabla_\theta \mathcal{L}(z_i;\theta^{(T)})}{\|\nabla_\theta \mathcal{L}(z_i;\theta^{(T)})\|}. \tag{69}$$

Since renormalized influence functions do not require estimating additional HVPs, it is considerably faster than RelatIF. Renormalized influence functions also do not have the additional error associated with estimating RelatIF's additional HVPs,[29]

This section should not be interpreted to mean that gradient magnitude is unimportant for influence analysis. On the contrary, gradient magnitude has a *significant* effect on training. However, the approximations made by existing influence estimators often overemphasize gradient magnitude leading to influence rankings that are not semantically meaningful.

Remark 9 Barshan et al. (2020) RelatIF and Hammoudeh & Lowd (2022) renormalization do not change the corresponding influence estimators' time and space complexities.

6 Applications of influence analysis

Sect. 3.3 discusses how a few topics (formally) relate to influence analysis. This section provides an extended discussion of influence analysis's applications. Specifically, this section focuses on higher-level learning tasks as opposed to the specific application environments where influence analysis has been used including: toxic speech detection (Han & Tsvetkov, 2021), social network graph labeling (Zhang et al., 2021c), user engagement detection (Liang et al., 2021), medical imaging annotation (Braun et al., 2022), etc.

First, *data cleaning* aims to improve a machine learning model's overall performance by removing "bad" training data. These "bad" instances arise due to disparate non-malicious causes including human/algorithmic labeling error, non-representative instances, noisy features, missing features, etc. (Krishnan et al., 2016; Liu et al., 2018; Krishnan & Wu, 2019). Intuitively, "bad" training instances are generally anomalous, and their features clash with the feature distribution of typical "clean" data (Wojnowicz et al., 2016). In practice, overparameterized neural networks commonly memorize these "bad" instances to achieve zero training loss (Hara et al., 2019; Feldman & Zhang, 2020; Pruthi et al., 2020; Thimonier et al., 2022). As explained in Sect. 3.2, memorization can be viewed as the influence of a training instance on itself. Therefore, influence analysis can be used to detect these highly memorized training instances. These memorized "bad" instances are then either removed from the training data or simply relabeled (Kong et al., 2022) and the model retrained.

[29] Hammoudeh & Lowd (2022) also provide renormalized versions of representer point and TracIn which are omitted here.

Poisoning and backdoor attacks craft malicious training instances that manipulate a model to align with some attacker objective. For example, a company may attempt to trick a spam filter so all emails sent by a competitor are erroneously classified as spam (Shafahi et al., 2018). Obviously, only influential (malicious) training instances affect a model's prediction. Some training set attacks rely on influence analysis to craft better (i.e., more influential) poison instances (Fang et al., 2020; Jagielski et al., 2021; Oh et al., 2022). Since most training set attacks do not assume the adversary knows training's random seed or even necessarily the target model's architecture, poison instances are crafted to maximize their expected group influence (Chen et al., 2017).

Influence and memorization analysis have also been used to improve *membership inference attacks*, where the adversary attempts to extract sensitive training data provided only a pretrained (language) model (Demontis et al., 2019; Cohen & Giryes, 2022).

Training set attack defenses detect and mitigate poisoning and backdoor attacks (Li et al., 2022). Since malicious training instances must be influential to achieve the attacker's objective, defending against adversarial attacks reduces to identifying abnormally influential training instances. If attackers are constrained in the number of training instances they can insert (Wallace et al., 2021; You et al., 2023), the target of a training set attack can be identified by searching for test instances that have a few exceptionally influential training instances (Hammoudeh & Lowd, 2022). The training set attack mitigation removes these anomalously influential instances from the training data and then retrains the model (Wang et al., 2019). In addition, influence estimation has been applied to the related task of *evasion attack detection*, where the training set is pristine and only test instances are perturbed (Cohen et al., 2020).

Algorithmic fairness promotes techniques that enable machine learning models to make decisions free of prejudices and biases based on inherited characteristics such as race, religion, and gender (Mehrabi et al., 2021). A classic example of model unfairness is the COMPAS software tool, which estimated the recidivism risk of incarcerated individuals. COMPAS was shown to be biased against black defendants, falsely flagging them as future criminals at twice the rate of white defendants (Angwin et al., 2016). Widespread adoption of algorithmic decision making in domains critical to human safety and well-being is predicated on the public's perception and understanding of the algorithms' inherent ethical principles and fairness (Awad et al., 2018). Yet, how to quantify the extent to which an algorithm is "fair" remains an area of active study (Dwork et al., 2012; Glymour & Herington, 2019; Saxena et al., 2019). Black & Fredrikson (2021) propose *leave-one-out unfairness* as a measure of a prediction's fairness. Intuitively, when a model's decision (e.g., not granting a loan, hiring an employee) is fundamentally changed by the inclusion of a *single* instance in a large training set, such a decision may be viewed as unfair or even capricious. Leave-one-out influence is therefore useful to measure and improve a model's robustness and fairness.

Explainability attempts to make a black-box model's decisions understandable by humans (Burkart & Huber, 2021). Transparent explanations are critical to achieving user trust of and satisfaction with ML systems (Lim et al., 2009; Kizilcec, 2016; Zhou et al., 2019). *Example-based explanations* communicate why a model made a particular prediction via visual examples (Cai et al., 2019; Sui et al., 2021)—e.g., training images—as social science research has shown that humans can understand complex ideas using only examples (Renkl et al., 2009; Renkl, 2014). Influence estimation can assist in the selection of canonical training instances that are particularly important for a given class in general or a single test prediction specifically. Similarly, *normative explanations*—which collectively establish a "standard" for a given class (Cai et al., 2019)—can be selected from

those training instances with the highest average influence on a held-out validation set. In cases where a test instance is misclassified, influence analysis can identify those training instances that most influenced the misprediction.

Subsampling reduces the computational requirements of large datasets by training models using only a subset of the training data (Ting & Brochu, 2018). Existing work has shown that high-quality training subsets can be created by greedily selecting training instances based on their overall influence (Khanna et al., 2019; Wang et al., 2020). Under mild assumptions, Wang et al. (2020) even show that, in expectation, influence-based subsampling performs at least as well as training on the full training set.

Annotating unlabeled data can be expensive—in particular for domains like medical imaging where the annotators must be domain experts Braun et al. (2022). Compared to labeling instances u.a.r., *active learning* reduces labeling costs by prioritizing annotation of particularly salient unlabeled data. In practice, active learning often simplifies to maximizing the *add-one-in influence* where each unlabeled instance's marginal influence must be estimated. Obviously, retraining for each possible unlabeled instance combination has exponential complexity and is intractable. Instead, a greedy strategy can be used where the influence of each unlabeled instance is estimated to identify the next candidate to label (Liu et al., 2021; Jia et al., 2021; Zhang et al., 2021c).

To enhance the benefit of limited labeled data, influence analysis has been used to create better augmented training data (Lee et al., 2020; Oh et al., 2021). These influence-guided *data augmentation* methods outperform traditional random augmentations, albeit with a higher computational cost.

7 Future directions

The trend of consistently increasing model complexity and opacity will likely continue for the foreseeable future. Simultaneously, there are increased societal and regulatory demands for algorithmic transparency and explainability. Influence analysis sits at the nexus of these competing trajectories (Zhou et al., 2019), which points to the field growing in importance and relevance. This section identifies important directions we believe influence analysis research should take going forward.

Emphasizing Group Influence over Pointwise Influence: Most existing methods target pointwise influence, which apportions credit for a prediction to training instances individually. However, for overparameterized models trained on large datasets, only the tails of the data distribution are heavily influenced by an individual instance (Feldman, 2020b). Instead, most predictions are moderately influenced by multiple training instances working in concert (Feldman & Zhang, 2020; Das et al., 2021; Basu et al., 2020).

As an additional complicating factor, pointwise influence within data-distribution modes is often approximately *supermodular* where the marginal effect of a training instance's deletion *increases* as more instances from a group are removed (Hammoudeh & Lowd, 2022). This makes pointwise influence a particularly poor choice for understanding most model behavior. To date, very limited work has systematically studied group influence (Koh et al., 2019; Basu et al., 2020; Hammoudeh & Lowd, 2022). Better group influence estimators could be immediately applied in various domains such as poisoning attacks, coreset selection, and model explainability.

Certified Influence Estimation: Certified defenses against poisoning and backdoor attacks guarantee that deleting a fixed number of instances from the training data will

not change a model's prediction (Steinhardt et al., 2017; Levine & Feizi, 2021; Jia et al., 2022; Wang et al., 2022; Hammoudeh & Lowd, 2023). These methods can be viewed as upper bounding the training data's group influence—albeit very coarsely. Most certified poisoning defenses achieve their bounds by leveraging "tricks" associated with particular model architectures [e.g., instance-based learners (Jia et al., 2022) and ensembles (Levine & Feizi, 2021; Wang et al., 2022; Hammoudeh & Lowd, 2024; Rezaei et al., 2023)] as opposed to a detailed analysis of a prediction's stability (Hammoudeh and Lowd, 2023). With limited exception (Jia et al., 2019a), today's influence estimators do not provide any meaningful guarantee of their accuracy. Rather, most influence estimates should be viewed as only providing—at best—guidance on an instance's "possible influence." Guaranteed or even probabilistic bounds on an instance's influence would enable influence estimation to be applied in settings where more than a "heuristic approximation" is required (Hammoudeh & Lowd, 2023).

Improved Scalability: Influence estimation is slow. Analyzing each training instance's influence on a single test instance can take several hours or more (Barshan et al., 2020; Kobayashi et al., 2020; Guo et al., 2021; Hammoudeh & Lowd, 2022). For influence estimation to be a practical tool, it must be at least an order of magnitude faster. Heuristic influence analysis speed-ups could prove very useful (Guo et al., 2021; Schioppa et al., 2022). However, the consequences (and limitations) of any empirical shortcuts need to be thoroughly tested, verified, and understood. Similarly, limited existing work has specialized influence methods to particular model classes (Jia et al., 2021) or data modalities (Yeh et al., 2022). While application-agnostic influence estimators are useful, their flexibility limits their scalability and accuracy. Both of these performance metrics may significantly improve via increased influence estimator specialization.

Surrogate Influence and Influence Transferability: An underexplored opportunity to improve influence analysis lies in the use of surrogate models (Sharchilev et al., 2018; Jia et al., 2019b, 2021; Brophy et al., 2023). For example, linear surrogates have proven quite useful for model explainability (Lundberg and Lee, 2017). While using only a model's linear layer as a surrogate may be "too reductive" (Yeh et al., 2022), it remains an open question whether other compact models remain an option. Any surrogate method must be accompanied by rigorous empirical evaluation to identify any risks and "blind spots" the surrogate may introduce (Rudin, 2019).

Increased Evaluation Diversity: Influence analysis has the capability to provide salient insights into why models behave as they do (Feldman & Zhang, 2020). As an example, Black and Fredrikson (2021) demonstrate how influence analysis can identify potential unfairness in an algorithmic decision. However, influence estimation evaluation is too often superficial and focuses on a very small subset of possible applications. For instance, most influence estimation evaluation focuses primarily on contrived data cleaning and mislabeled training data experiments (Wojnowicz et al., 2016; Koh & Liang, 2017; Khanna et al., 2019; Ghorbani & Zou, 2019; Yeh et al., 2018; Pruthi et al., 2020; Chen et al., 2021; Terashita et al., 2021); K & Søgaard, 2021; (Sui et al., 2021; Brophy et al., 2023; Yeh et al., 2022; Kong et al., 2022; Kwon and Zou, 2022). It is unclear how these experiments translate into real-world or adversarial settings, with recent work pointing to generalization fragility (Basu et al., 2021; Bae et al., 2022; Schioppa et al., 2023). We question whether these data cleaning experiments—where specialized methods already exist (Krishnan et al., 2016; Krishnan & Wu, 2019; Wang et al., 2019)—adequately satisfy influence analysis's stated promise of providing "understanding [of] black-box predictions" (Koh & Liang, 2017).

Objective Over Subjective Evaluation Criteria: A common trope when evaluating an influence analysis method is to provide a test example and display training instances the estimator

identified as most similar or dissimilar. These "eye test" evaluations are generally applied to vision datasets (Koh & Liang, 2017; Yeh et al., 2018; Jia et al., 2019a; Pruthi et al., 2020; Feldman & Zhang, 2020) and to a limited extent other modalities. Such experiments are unscientific. They provide limited meaningful insight given the lack of a ground truth by which to judge the results. Most readers do not have detailed enough knowledge of a dataset to know whether the selected instances are especially representative. Rather, there may exist numerous training instances that are much more similar to the target that the influence estimator overlooked. Moreover, such visual assessments are known to be susceptible to confirmation and expectancy biases (Mahoney, 1977; Nakhaeizadeh et al., 2013; Kassin et al., 2013).

Influence analysis evaluation should focus on experiments that are quantifiable and verifiable w.r.t. a ground truth.

8 Conclusions

While influence analysis has received increased attention in recent years, significant progress remains to be made. Influence estimation is computationally expensive and can be prone to inaccuracy. Going forward, fast certified influence estimators are needed. Nonetheless, despite these shortcomings, existing applications already demonstrate influence estimation's capabilities and promise.

This work reviews numerous methods with different perspectives on—and even definitions of—training data influence. It would be a mistake to view this diversity of approaches as a negative. While no single influence analysis method can be applied to all situations, most use cases should have at least one method that fits well. An obvious consequence then is the need for researchers and practitioners to understand the strengths and limitations of the various methods so as to know which method best fits their individual use case. This survey is intended to provide that insight from both empirical and theoretical viewpoints.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s10994-023-06495-7.

Author contributions ZH conceptualized the taxonomy's structure and organization of the survey. ZH selected the survey's critical related references with input from DL. ZH wrote the manuscript with input from DL. DL supervised the project.

Funding This work was supported by a grant from the Air Force Research Laboratory and the Defense Advanced Research Projects Agency (DARPA) — agreement number FA8750-16-C-0166, subcontract K001892-00-S05, as well as a second grant from DARPA, agreement number HR00112090135.

Declarations

Conflict of interest All authors certify that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter or materials discussed in this manuscript.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Agarwal, N., Bullins, B., & Hazan, E. (2017). Second-order stochastic optimization for machine learning in linear time. *Journal of Machine Learning Research, 18*(1), 4148–4187.

Aha, D. W., Kibler, D., & Albert, M. K. (1991). Instance-based learning algorithms. *Machine Learning, 6*(1), 37–66.

Angwin, J., Larson, J., Mattu, S., & Kirchner, L. (2016). Machine bias. ProPublica. https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing

Arnoldi, W. E. (1951). The principle of minimized iterations in the solution of the matrix eigenvalue problem. *Quarterly of Applied Mathematics, 9*(1), 17–29.

Arpit, D., Jastrzebski, S., Ballas, N., Krueger, D., Bengio, E., Kanwal, M.S., Maharaj, T., Fischer, A., Courville, A., Bengio, Y., & Lacoste-Julien, S. (2017). A closer look at memorization in deep networks. In *Proceedings of the 34th international conference on machine learning. ICML'17*., arXiv:1706.05394

Awad, E., Dsouza, S., Kim, R., Schulz, J., Henrich, J., Shariff, A., Bonnefon, J. F., & Rahwan, I. (2018). The moral machine experiment. *Nature, 563*(7729), 59–64.

Bachem, O., Lucic, M., & Krause, A. (2017). Practical coreset constructions for machine learning

Bae, J., Ng, N., Lo, A., Ghassemi, M., & Grosse, R. (2022). If influence functions are the answer, then what is the question? In *Proceedings of the 36th Conference on Neural Information Processing Systems. NeurIPS'22, Curran Associates, Inc*. arXiv:2209.05364

Banzhaf, J. F. I. (1965). Weighted voting doesn't work: A mathematical analysis. *Rutgers Law Review, 19*(2), 317–343.

Barshan, E., Brunet, M., Dziugaite, G.K. (2020). RelatIF: Identifying explanatory training samples via relative influence. In *Proceedings of the 23rd International conference on artificial intelligence and statistics*. AISTATS'20. arXiv:2003.11630

Bartlett, P. L., Long, P. M., Lugosi, G., & Tsigler, A. (2020). Benign overfitting in linear regression. *Proceedings of the National Academy of Sciences, 117*(48), 30063–30070.

Basta, C., Costa-jussà, M.R., & Casas, N. (2019). Evaluating the underlying gender bias in contextualized word embeddings. In *Proceedings of the first workshop on gender bias in natural language processing. Association for computational linguistics*. arXiv:1904.08783

Basu, S., Pope, P., & Feizi, S. (2021). Influence functions in deep learning are fragile. In *Proceedings of the 9th International Conference on Learning Representations. ICLR'21, Virtual Only*. arXiv:2006.14651

Basu, S., You, X., & Feizi, S. (2020). On second-order group influence functions for black-box predictions. In *Proceedings of the 37th International conference on machine learning. ICML'20, PMLR, Virtual Only*. arXiv:1911.00418

Beaton, A. E., & Tukey, J. W. (1974). The fitting of power series, meaning polynomials, illustrated on band-spectroscopic data. *Technometrics, 16*(2), 147–185.

Belle, V., & Papantonis, I. (2021). Principles and practice of explainable machine learning. *Frontiers in Big Data, 4*, 688969–688969.

Bender, E.M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*. p. 610–623. FAccT'21, Association for Computing Machinery. https://doi.org/10.1145/3442188.3445922

Biggio, B., Nelson, B., Laskov, P. (2012). Poisoning attacks against support vector machines. In *Proceedings of the 29th international conference on machine learning. ICML'12, PMLR, Edinburgh, Great Britain*, arXiv:1206.6389

Bilmes, J. (2022) Submodularity in machine learning and artificial intelligence , arXiv:2202.00132

Black, E., & Fredrikson, M. (2021). Leave-one-out unfairness. In *Proceedings of the 2021 acm conference on fairness, accountability, and transparency. FAccT'2*. arXiv:2107.10171

Blum, A. L., & Rivest, R. L. (1992). Training a 3-node neural network is NP-complete. *Neural Networks, 5*(1), 117–127.

Bordt, S., & von Luxburg, U. (2023). From Shapley values to generalized additive models and back. In *Proceedings of The 26th international conference on artificial intelligence and statistics. AISTATS'23* (2023) , arXiv:2209.04012

Borsos, Z., Mutny, M., Krause, A. (2020). Coresets via bilevel optimization for continual learning and streaming. In *Proceedings of the 34th conference on neural information processing systems. NeurIPS'20*. arXiv:2006.03875

Boyd, S., & Vandenberghe, L. (2004). *Convex optimization*. Cambridge University Press.

Braun, J., Kornreich, M., Park, J., Pawar, J., Browning, J., Herzog, R., Odry, B., & Zhang, L. (2022) Influence based re-weighing for labeling noise in medical imaging. In *Proceedings of the 19th IEEE International Symposium on Biomedical Imaging*. ISBI'22

Brophy, J., Hammoudeh, Z., & Lowd, D. (2023). Adapting and evaluating influence-estimation methods for gradient-boosted decision trees. *Journal of Machine Learning Research, 24*, 1–48.

Brophy, J., & Lowd, D. (2021). Machine unlearning for random forests. In *Proceedings of the 38th international conference on machine learning*. ICML'21. arXiv:2009.05567

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., & Amodei, D. (2020). Language models are few-shot learners. In *Proceedings of the 34th conference on neural information processing systems*. NeurIPS'20, Curran Associates, Inc. arXiv:2005.14165

Burkart, N., & Huber, M. F. (2021). A survey on the explainability of supervised machine learning. *Journal Artificial Intelligence Research, 70*, 245–317.

Cai, C.J., Jongejan, J., & Holbrook, J. (2019). The effects of example-based explanations in a machine learning interface. In *Proceedings of the 24th international conference on intelligent user interfaces*. p. 258–262. IUI'19 . https://doi.org/10.1145/3301275.3302289

Chen, R., Li, Z., Li, J., Wu, C., Yan, J. (2022). On collective robustness of bagging against data poisoning. In *Proceedings of the 39th international conference on machine learning*. ICML'22, PMLR. arXiv:2205.13176

Chen, X., Liu, C., Li, B., Lu, K., Song, D. (2017). Targeted backdoor attacks on deep learning systems using data poisoning. arXiv:1712.05526

Chen, Y., Li, B., Yu, H., Wu, P., & Miao, C. (2021). HyDRA: Hypergradient data relevance analysis for interpreting deep neural networks. In *Proceedings of the 35th AAAI conference on artificial intelligence. AAAI'21, association for the advancement of artificial intelligence, virtual only*. arXiv:2102.02515

Cohen, G., & Giryes, R. (2022). Membership inference attack using self influence functions. arXiv:2205.13680

Cohen, G., Sapiro, G., & Giryes, R. (2020) Detecting adversarial samples using influence functions and nearest neighbors. In *Proceedings of the IEEE conference on computer vision and pattern recognition. CVPR'20, Virtual Only*. arXiv:1909.06872

Cook, R. D. (1977). Detection of influential observation in linear regression. *Technometrics, 19*(1), 15–18.

Cook, R. D., Holschuh, N., & Weisberg, S. (1982). A note on an alternative outlier model. *Journal of the Royal Statistical Society Series B (Methodological), 44*(3), 370–376.

Cook, R. D., & Weisberg, S. (1982). *Residuals and influence in regression*. Chapman and Hall.

Dai, Z., & Gifford, D.K. (2023). Training data attribution for diffusion models. arXiv:2306.02174

D'Amour, A., Heller, K.A., Moldovan, D., Adlam, B., Alipanahi, B., Beutel, A., Chen, C., Deaton, J., Eisenstein, J., Hoffman, M.D., Hormozdiari, F., Houlsby, N., Hou, S., Jerfel, G., Karthikesalingam, A., Lucic, M., Ma, Y., McLean, C.Y., Mincu, D., Mitani, A., Montanari, A., Nado, Z., Natarajan, V., Nielson, C., Osborne, T.F., Raman, R., Ramasamy, K., Sayres, R., Schrouff, J., Seneviratne, M., Sequeira, S., Suresh, H., Veitch, V., Vladymyrov, M., Wang, X., Webster, K., Yadlowsky, S., Yun, T., Zhai, X., & Sculley, D. (2020). Underspecification presents challenges for credibility in modern machine learning. arXiv:2011.03395

Das, S., Singh, A., Chatterjee, S., Bhattacharya, S., & Bhattacharya, S. (2021). Finding high-value training data subset through differentiable convex programming. In *Proceedings of the 2021 European conference on machine learning and principles and practice of knowledge discovery in databases. ECML PKDD'21*. arXiv:2104.13794

Davies, A., & Ghahramani, Z. (2014) The random forest kernel and other kernels for big data from random partitions. arXiv:1402.4293

Demontis, A., Melis, M., Pintor, M., Jagielski, M., Biggio, B., Oprea, A., Nita-Rotaru, C., & Roli, F. (2019). Why do adversarial attacks transfer? Explaining transferability of evasion and poisoning attacks. In: *Proceedings of the 28th USENIX security symposium. USENIX'19*. arXiv:1809.02861

Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., & Fei-Fei, L. (2009) ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. pp. 248–255. CVPR'09.

Deng, X., & Papadimitriou, C. H. (1994). On the complexity of cooperative solution concepts. *Mathematics of Operations Research, 19*(2), 257–266.

Devlin, J., Chang, M.W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics*. ACL'19, Association for Computational Linguistics, Minneapolis, Minnesota. arXiv:1810.04805.

Diskin, M., Bukhtiyarov, A., Ryabinin, M., Saulnier, L., Lhoest, Q., Sinitsin, A., Popov, D., Pyrkin, D., Kashirin, M., Borzunov, A., del Moral, A.V., Mazur, D., Kobelev, I., Jernite, Y., Wolf, T., & Pekhimenko, G. (2021). Distributed deep learning in open collaborations. In *Proceedings of the 35th conference on neural information processing Systems. NeurIPS'21*. arXiv:2106.10207

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2019). An image is worth 16x16 words: Transformers for image recognition at scale. In *Proceedings of the 9th international conference on learning representations*. ICLR'21, Virtual Only. arXiv:2010.11929

Dubey, P. (1975). On the uniqueness of the Shapley Value. *International Journal of Game Theory, 4*(3), 131–139.

Dubey, P., Neyman, A., & Weber, R. J. (1981). Value theory without efficiency. *Mathematics of Operations Research, 6*(1), 122–128.

Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research, 12*, 2121–2159.

Dwork, C., Hardt, M., Pitassi, T., Reingold, O., & Zemel, R. (2012). Fairness through awareness. In *Proceedings of the 3rd innovations in theoretical computer science conference. ITCS'12*. arXiv:1104.3913

Eisenhofer, T., Riepel, D., Chandrasekaran, V., Ghosh, E., Ohrimenko, O., & Papernot, N. (2022) Verifiable and provably secure machine unlearning. arXiv:2210.09126

Ekambaram, R., Goldgof, D.B., & Hall, L.O. (2017). Finding label noise examples in large scale datasets. In *Proceeding of the 2017 IEEE international conference on systems, man, and cybernetics*.

Ellenberg, J. H. (1976). Testing for a single outlier from a general linear regression. *Biometrics, 32*(3), 637–645.

Fang, M., Gong, N.Z., & Liu, J. (2020). Influence function based data poisoning attacks to top-N recommender systems. In: Proceedings of the Web Conference 2020. WWW'20. arXiv:2002.08025

Feldman, D. (2020a). Introduction to core-sets: an updated survey. arXiv:2011.09384.

Feldman, V. (2020b). Does learning require memorization? A short tale about a long tail. In *Proceedings of the 52nd Annual ACM SIGACT symposium on theory of computing*. STOC'20. arXiv:1906.05271

Feldman, V., & Zhang, C. (2020). What neural networks memorize and why: Discovering the long tail via influence estimation. In *Proceedings of the 34th conference on neural information processing systems*. NeurIPS'20, Curran Associates, Inc., Virtual Only. arXiv:2008.03703

Fowl, L., Goldblum, M., Chiang, P., Geiping, J., Czaja, W., & Goldstein, T. (2021). Adversarial examples make strong poisons. In *Proceedings of the 35th conference on neural information processing systems*. NeurIPS'21, Curran Associates, Inc., Virtual Only. arXiv:2106.10807

Ghorbani, A., & Zou, J. (2019). Data Shapley: Equitable valuation of data for machine learning. In *Proceedings of the 36th international conference on machine learning*. ICML'19 (2019) , https://proceedings.mlr.press/v97/ghorbani19c.html

Ghorbani, A., & Zou, J.Y. (2020). Neuron Shapley: Discovering the responsible neurons. In *Proceedings of the 34th conference on neural information processing systems*. NeurIPS'20. arXiv:2002.09815

Glymour, B., & Herington, J. (2019). Measuring the biases that matter: The ethical and casual foundations for measures of fairness in algorithms. In *Proceedings of the 2019 ACM conference on fairness, accountability, and transparency*. FAccT'19.

Goyal, P., Dollár, P., Girshick, R.B., Noordhuis, P., Wesolowski, L., Kyrola, A., Tulloch, A., Jia, Y., He, K. (2017). Accurate, large minibatch SGD: Training ImageNet in 1 hour. arXiv:1706.02677

Grabisch, M., & Roubens, M. (1999). An axiomatic approach to the concept of interaction among players in cooperative games. *International Journal of Game Theory, 28*(4), 547–565. https://doi.org/10.1007/s001820050125

Guo, C., Goldstein, T., Hannun, A.Y., & van der Maaten, L. (2020). Certified data removal from machine learning models. In *Proceedings of the 37th international conference on machine learning. ICML'20*, vol. 119, pp. 3832–3842. arXiv:1911.03030

Guo, H., Rajani, N., Hase, P., Bansal, M., & Xiong, C. (2021). FastIF: Scalable influence functions for efficient model interpretation and debugging. In *Proceedings of the 2021 conference on empirical methods in natural language processing. EMNLP'21*. arXiv:2012.15781.

Hammoudeh, Z., & Lowd, D. (2021). Simple, attack-agnostic defense against targeted training set attacks using cosine similarity. In *Proceedings of the 3rd ICML workshop on uncertainty and robustness in deep learning. UDL'21*.

Hammoudeh, Z., & Lowd, D. (2022). Identifying a training-set attack's target using renormalized influence estimation. In *Proceedings of the 29th ACM SIGSAC conference on computer and communications security*. CCS'22, Association for Computing Machinery. arXiv:2201.10055.

Hammoudeh, Z., & Lowd, D. (2023). Reducing certified regression to certified classification for general poisoning attacks. In *Proceedings of the 1st IEEE conference on secure and trustworthy machine learning. SaTML'23*. arXiv:2208.13904.

Hammoudeh, Z., & Lowd, D. (2024) Provable robustness against a union of ℓ_0 attacks. In *Proceedings of the 38th AAAI conference on artificial intelligence. AAAI'24*. arXiv:2302.11628.

Hampel, F. R. (1974). The influence curve and its role in robust estimation. *Journal of the American Statistical Association, 69*(346), 383–393.

Han, X., & Tsvetkov, Y. (2021). Fortifying toxic speech detectors against veiled toxicity. In *Proceedings of the 2020 conference on empirical methods in natural language processing. EMNLP'20*. arXiv:2010.03154.

Hara, S., Nitanda, A., & Maehara, T. (2019) Data cleansing for models trained with SGD. In *Proceedings of the 33rd conference on neural information processing systems*. NeurIPS'19, Curran Associates, Inc. arXiv:1906.08473

He, X., Pan, J., Jin, O., Xu, T., Liu, B., Xu, T., Shi, Y., Atallah, A., Herbrich, R., Bowers, S., & Candela, J.Q.N. (2014). Practical lessons from predicting clicks on ads at Facebook. In *Proceedings of the eighth international workshop on data mining for online advertising. AdKDD'14*. Association for Computing Machinery. https://research.facebook.com/publications/practical-lessons-from-predicting-clicks-on-ads-at-facebook/.

Higgins, I., Matthey, L., Pal, A., Burgess, C.P., Glorot, X., Botvinick, M.M., Mohamed, S., & Lerchner, A. (2017). beta-VAE: Learning basic visual concepts with a constrained variational framework. In *Proceedings of the 5th international conference on learning representations. ICLR'17*. https://openreview.net/forum?id=Sy2fzU9gl.

Hofmann, T., Schölkopf, B., & Smola, A. J. (2008). Kernel methods in machine learning. *Annals of Statistics, 36*(3), 1171–1220. arXiv:math/0701907.

Hogg, R. V. (1979). Statistical robustness: One view of its use in applications today. *The American Statistician, 33*(3), 108–115.

Huber, P. (1981). *Robust statistics*. Wiley.

Huber, P. J. (1964). Robust estimation of a location parameter. *Annals of Mathematical Statistics, 35*(1), 73–101.

Hutchinson, B., Prabhakaran, V., Denton, E., Webster, K., Zhong, Y., & Denuyl, S. (2020). Social biases in NLP models as barriers for persons with disabilities. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. Association for Computational Linguistics.

Ilyas, A., Park, S.M., Engstrom, L., Leclerc, G., & Madry, A. (2022). Datamodels: Understanding predictions with data and data with predictions. In *Proceedings of the 39th international conference on machine learning*. ICML'22, PMLR. arXiv:2202.00622

Jaeckel, L. A. (1972). *The infinitesimal jackknife, Tech. rep.* Bell Laboratories.

Jagielski, M., Severi, G., Pousette Harger, N., & Oprea, A. (2021) Subpopulation data poisoning attacks. In *Proceedings of the 28th ACM SIGSAC conference on computer and communications security. ccs '21, association for computing machinery, Virtual Only*. arXiv:2006.14026

Jia, J., Liu, Y., Cao, X., & Gong, N.Z. (2022). Certified robustness of nearest neighbors against data poisoning and backdoor attacks. In *Proceedings of the 36th AAAI conference on artificial intelligence. AAAI'22*. arXiv:2012.03765.

Jia, R., Dao, D., Wang, B., Hubis, F.A., Gürel, N.M., Li, B., Zhang, C., Spanos, C.J., & Song, D. (2019a). Efficient task-specific data valuation for nearest neighbor algorithms. In *Proceedings of the VLDB endowment. PVLDB'19*. arXiv:1908.08619

Jia, R., Dao, D., Wang, B., Hubis, F.A., Hynes, N., Gürel, N.M., Li, B., Zhang, C., Song, D., & Spanos, C.J. (2019b). Towards efficient data valuation based on the Shapley value. In *Proceedings of the 22nd conference on artificial intelligence and statistics*. pp. 1167–1176. AISTATS'19. arXiv:1902.10275

Jia, R., Wu, F., Sun, X., Xu, J., Dao, D., Kailkhura, B., Zhang, C., Li, B., & Song, D. (2021). Scalability vs. utility: Do we have to sacrifice one for the other in data importance quantification? In *Proceedings of the 2021 IEEE/CVF conference on computer vision and pattern recognition*. CVPR'21 (2021) , arXiv:1911.07128

Jiang, Z., Zhang, C., Talwar, K., & Mozer, M.C. (2021). Characterizing structural regularities of labeled data in overparameterized models. In *Proceedings of the 38th international conference on machine learning*. (pp. 5034–5044). ICML'21. arXiv:2002.03206

Johnson, W. B., & Lindenstrauss, J. (1984). Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics, 26*, 189–206.

Dennis Jr., J. E., & Welsch, R. E. (1978). Techniques for nonlinear least squares and robust regression. *Communications in Statistics-Simulation and Computation, 7*(4), 345–359.

K, K., & Søgaard, A. (2021). Revisiting methods for finding influential examples. arXiv:2111.04683.

Kandpal, N., Wallace, E., & Raffel, C. (2022). Deduplicating training data mitigates privacy risks in language models. In *Proceedings of the 39th international conference on machine learning*. ICML'22. arXiv:2202.06539

Kassin, S. M., Dror, I. E., & Kukucka, J. (2013). The forensic confirmation bias: Problems, perspectives, and proposed solutions. *Journal of Applied Research in Memory and Cognition, 2*(1), 42–52.

Khanna, R., Kim, B., Ghosh, J., Koyejo, O. (2019). Interpreting black box predictions using Fisher kernels. In *Proceedings of the 22nd conference on artificial intelligence and statistics. AISTATS'19*. arXiv:1810.10118.

Ki, N., Choi, H., Chung, H.W. (2023). Data valuation without training of a model. In *Proceedings of the 11th international conference on learning representations. ICLR'23*. https://openreview.net/forum?id=XIzO8zr-WbM.

Kingma, D.P., & Ba, J. (2021). Adam: A method for stochastic optimization. In *Proceedings of the 3rd international conference on learning representations*. ICLR'15. arXiv:1412.6980

Kingma, D.P., & Welling, M. (2014) Auto-encoding variational Bayes. In *Proceedings of the 2nd international conference on learning representations*. ICLR'14 (2014). arXiv:1312.6114.

Kizilcec, R.F. (2016). How much information? Effects of transparency on trust in an algorithmic interface. In *Proceedings of the 2016 CHI conference on human factors in computing systems*, (pp. 2390–2395). CHI'16, Association for Computing Machinery.

Knight, W. (2017). The dark secret at the heart of AI. MIT Technology Review. https://www.technologyreview.com/2017/04/11/5113/the-dark-secret-at-the-heart-of-ai/.

Kobayashi, S., Yokoi, S., Suzuki, J., & Inui, K. (2020). Efficient estimation of influence of a training instance. In *Proceedings of SustaiNLP: Workshop on Simple and Efficient Natural Language Processing*. Association for Computational Linguistics. arXiv:2012.04207.

Koh, P.W., Ang, K.S., Teo, H.H.K., & Liang, P. (2019). On the accuracy of influence functions for measuring group effects. In *Proceedings of the 33rd international conference on neural information processing systems*. NeurIPS'19, Curran Associates Inc., arXiv:1905.13289

Koh, P.W., & Liang, P. (2017). Understanding black-box predictions via influence functions. In *Proceedings of the 34th international conference on machine learning*. ICML'17, PMLR. arXiv:1703.04730.

Kong, S., Shen, Y., & Huang, L. (2022). Resolving training biases via influence-based data relabeling. In *Proceedings of the 10th international conference on learning representations*, ICLR'22. https://openreview.net/forum?id=EskfH0bwNVn.

Kong, Z., & Chaudhuri, K. (2021) Understanding instance-based interpretability of variational auto-encoders. In *Proceedings of the 35th conference on neural information processing systems*. NeurIPS'21, Curran Associates, Inc. arXiv:2105.14203.

Krishnan, S., Wang, J., Wu, E., Franklin, M.J., & Goldberg, K. (2016). ActiveClean: Interactive data cleaning for statistical modeling. In *Proceedings of the VLDB endowment*. https://www.vldb.org/pvldb/vol9/p948-krishnan.pdf.

Krishnan, S., & Wu, E. (2019). AlphaClean: Automatic generation of data cleaning pipelines. arXiv:1904.11827.

Krizhevsky, A., Nair, V., & Hinton, G. (2014). The CIFAR-10 dataset.

Krizhevsky, A., Sutskever, I., & Hinton, G.E. (2012). ImageNet classification with deep convolutional neural networks. In *Proceedings of the 25th conference on neural information processing systems*. (pp. 1097–1105). NeurIPS'12.

Kurita, K., Vyas, N., Pareek, A., Black, A.W., & Tsvetkov, Y. (2019). Measuring bias in contextualized word representations. In *Proceedings of the first workshop on gender bias in natural language processing*. Association for Computational Linguistics. arXiv:1906.07337

Kwon, Y., & Zou, J. (2022). Beta Shapley: A unified and noise-reduced data valuation framework for machine learning. In *Proceedings of the 25th conference on artificial intelligence and statistics*. AISTATS'22, PMLR. arXiv:2110.14049

Leclerc, Y. G. (1989). Constructing simple stable descriptions for image partitioning. *International Journal of Computer Vision, 3*(1), 73–102.

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. In *Proceedings of the IEEE*. (vol. 86, pp. 2278–2324).

Lee, D., Park, H., Pham, T., & Yoo, C.D. (2020). Learning augmentation network via influence functions. In *Proceedings of the 33rd conference on computer vision and pattern recognition*. CVPR'20.

Levine, A., & Feizi, S. (2021). Deep partition aggregation: Provable defenses against general poisoning attacks. In *Proceedings of the 9th international conference on learning representations*. ICLR'21, Virtual Only. arXiv:2006.14768.

Li, Y., Wu, B., Jiang, Y., Li, Z., & Xia, S. (2022). Backdoor learning: A survey. In *IEEE transactions on neural networks and learning systems*. arXiv:2007.08745

Liang, W., Liang, K.H., & Yu, Z. (2021). HERALD: An annotation efficient method to detect user disengagement in social conversations. In *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing*. ACL-IJCNLP'21, Association for Computational Linguistics. arXiv:2106.00162.

Lim, B.Y., Dey, A.K., & Avrahami, D. (2009). Why and why not explanations improve the intelligibility of context-aware intelligent systems. In *Proceedings of the SIGCHI conference on human factors in computing systems*. (p. 2119–2128). CHI'09, Association for Computing Machinery.

Lin, J., Zhang, A., Lecuyer, M., Li, J., Panda, A., & Sen, S. (2022). Measuring the effect of training data on deep learning predictions via randomized experiments. In *Proceedings of the 39th international conference on machine learning*. ICML'22. arXiv:2206.10013.

Lipton, Z. C. (2018). The mythos of model interpretability: In machine learning, the concept of interpretability is both important and slippery. *Queue, 16*(3), 31–57. https://doi.org/10.1145/3236386.3241340

Liu, K., Dolan-Gavitt, B., & Garg, S. (2018). Fine-pruning: Defending against backdooring attacks on deep neural networks. In *Proceedings of the international symposium on research in attacks, intrusions, and defenses*, (pp. 273–294). RAID'18, Springer. arXiv:1805.12185.

Liu, Z., Ding, H., Zhong, H., Li, W., Dai, J., & He, C. (2021). Influence selection for active learning. In *Proceedings of the 18th international conference on computer vision*. ICCV'21. arXiv:2108.09331.

Lundberg, S.M., & Lee, S.I. (2017). A unified approach to interpreting model predictions. In *Proceedings of the 31st international conference on neural information processing systems*. NeurIPS'17 (2017) , arXiv:1705.07874.

Mahoney, M. J. (1977). Publication prejudices: An experimental study of confirmatory bias in the peer review system. *Cognitive Therapy and Research, 1*(2), 161–175.

Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A survey on bias and fairness in machine learning. *ACM Computing Surveys, 54*(6), 1–35.

Mirzasoleiman, B., Bilmes, J., & Leskovec, J. (2020). Coresets for data-efficient training of machine learning models. In *Proceedings of the 37th international conference on machine learning*. ICML'20. arXiv:1906.01827.

Nakhaeizadeh, S., Dror, I. E., & Morgan, R. M. (2013). Cognitive bias in forensic anthropology: Visual assessment of skeletal remains is susceptible to confirmation bias. *Science & Justice, 54*(3), 208–214.

Negahban, S. N., Ravikumar, P., Wainwright, M. J., & Yu, B. (2012). A unified framework for high-dimensional analysis of *M*-estimators with decomposable regularizers. *Statistical Science, 27*(4), 538–557.

Nguyen, E., Seo, M., & Oh, S.J. (2023). A Bayesian perspective on training data attribution. arXiv:2305.19765.

Nguyen, T. T., Huynh, T. T., Nguyen, P. L., Liew, A. W. C., Yin, H., & Nguyen, Q. V. H. (2022). *A survey of machine unlearning., 2209*, 02299.

Oh, S., Kim, S., Rossi, R.A., & Kumar, S. (2021). Influence-guided data augmentation for neural tensor completion. In *Proceedings of the 30th ACM international conference on information and knowledge management*. CIKM'21, ACM. arXiv:2108.10248.

Oh, S., Ustun, B., McAuley, J., & Kumar, S. (2022). Rank list sensitivity of recommender systems to interaction perturbations. In *Proceedings of the 31st ACM international conference on information and knowledge management*. CIKM'22, ACM. arXiv:2201.12686.

Park, S.M., Georgiev, K., Ilyas, A., Leclerc, G., & Madry, A. (2023). TRAK: Attributing model behavior at scale. In *Proceedings of the 40th international conference on machine learning*. ICML'23. arXiv:2303.14186

Pearlmutter, B. A. (1994). Fast exact multiplication by the Hessian. *Neural Computation, 6*, 147–160.

Pleiss, G., Zhang, T., Elenberg, E., & Weinberger, K.Q. (2020). Identifying mislabeled data using the area under the margin ranking. In *Proceedings of the 34th international conference on neural information processing systems*. NeurIPS'20, Curran Associates Inc., Red Hook. arXiv:2001.10528.

Pruthi, G., Liu, F., Kale, S., & Sundararajan, M. (2020) Estimating training data influence by tracing gradient descent. In *Proceedings of the 34th conference on neural information processing systems*. NeurIPS'20, Curran Associates, Inc., Virtual Only. arXiv:2002.08484

Qian, N. (1999). On the momentum term in gradient descent learning algorithms. *Neural Networks, 12*(1), 145–151.

Raste, S., Singh, R., Vaughan, J., & Nair, V.N. (2022). Quantifying inherent randomness in machine learning algorithms. arXiv:2206.12353

Recht, B. (2011). A simpler approach to matrix completion. *Journal of Machine Learning Research, 12*, 3413–3430.

Reddi, V. J., Diamos, G., Warden, P., Mattson, P., & Kanter, D. (2021). *Data engineering for everyone.* (vol. 2102, p. 11447). arXiv:2102.11447

Ren, P., Xiao, Y., Chang, X., Huang, P. Y., Li, Z., Gupta, B. B., Chen, X., & Wang, X. (2021). A survey of deep active learning. *ACM Computing Surveys, 54*(9), 1–40.

Renkl, A. (2014). Toward an instructionally oriented theory of example-based learning. *Cognitive Science, 38*(1), 1–37.

Renkl, A., Hilbert, T., & Schworm, S. (2009). Example-based learning in heuristic domains: A cognitive load theory account. *Educational Psychology Review, 21*(1), 67–78.

Rezaei, K., Banihashem, K., Chegini, A., & Feizi, S. (2023). Run-off election: Improved provable defense against data poisoning attacks. In *Proceedings of the 40th international conference on machine learning*. ICML'23. arXiv:2302.02300

Rezende, D.J., Mohamed, S., & Wierstra, D. (2014). Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of the 31st international conference on international conference on machine learning*. ICML'14. arXiv:1401.4082.

Rousseeuw, P. (1994). Least median of squares regression. *Journal of the American Statistical Association, 79*(388), 871–880.

Rousseeuw, P. J., & Leroy, A. M. (1987). *Robust regression and outlier detection*. Wiley.

Rozemberczki, B., Watson, L., Bayer, P., Yang, H.T., Kiss, O., Nilsson, S., & Sarkar, R. (2022). The Shapley value in machine learning. arXiv:2202.05594.

Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence, 1*(5), 206–215.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature, 323*(6088), 533–536.

Saxena, N.A., Huang, K., DeFilippis, E., Radanovic, G., Parkes, D.C., & Liu, Y. (2019). How do fairness definitions fare? Examining public attitudes towards algorithmic definitions of fairness. In *Proceedings of the 2019 AAAI/ACM conference on AI, ethics, and society*. AIES'19. arXiv:1811.03654.

Schioppa, A., Filippova, K., Titov, I., & Zablotskaia, P. (2023). Theoretical and practical perspectives on what influence functions do. arXiv:2305.16971.

Schioppa, A., Zablotskaia, P., Torres, D.V., & Sokolov, A. (2022) Scaling up influence functions. In *Proceedings of the 36th AAAI conference on artificial intelligence*. AAAI'22. arXiv:2112.03052

Schölkopf, B., Herbrich, R., & Smola, A.J. (2001). A generalized representer theorem. In *Proceedings of the 14th annual conference on computational learning theory and 5th european conference on computational learning theory*. (pp. 416–426). COLT'01/EuroCOLT'01, Springer.

Shafahi, A., Huang, W.R., Najibi, M., Suciu, O., Studer, C., Dumitras, T., & Goldstein, T. (2018) Poison frogs! Targeted clean-label poisoning attacks on neural networks. In *Proceedings of the 32nd Conference on Neural Information Processing Systems*. NeurIPS'18, Curran Associates, Inc. arXiv:1804.00792.

Shapley, L. S. (1953). A value for n-person games. In H. W. Kuhn & A. W. Tucker (Eds.), *Contributions to the Theory of Games II* (pp. 307–317). Princeton University Press.

Shapley, L. S., & Roth, A. E. (1988). *The Shapley Value: Essays in Honor of Lloyd S*. Shapley: Cambridge University Press.

Sharchilev, B., Ustinovskiy, Y., Serdyukov, P., & de Rijke, M. (2018). Finding influential training samples for gradient boosted decision trees. In *Proceedings of the 35th international conference on machine learning*, (pp. 4577–4585). ICML'18, PMLR. arXiv:1802.06640

Snedecor, W. G., & Cochran, G. W. (1968). *Statistical Methods* (6th ed.). Iowa State University Press.

Srikantan, K. S. (1961). Testing for the single outlier in a regression model. *Indian Journal of Statistics, 23*(3), 251–260.

Steinhardt, J., Koh, P.W., & Liang, P. (2017) Certified defenses for data poisoning attacks. In *Proceedings of the 31st conference on neural information processing systems*. NeurIPS'17, Curran Associates, Inc., Long Beach.arXiv:1706.03691.

Strubell, E., Ganesh, A., & McCallum, A. (2020). Energy and policy considerations for modern deep learning research. In *Proceedings of the 34th AAAI conference on artificial intelligence*. AAAI'20. arXiv:1906.02243.

Sui, Y., Wu, G., & Sanner, S. (2021). Representer point selection via local Jacobian expansion for post-hoc classifier explanation of deep neural networks and ensemble models. In *Proceedings of the 35th conference on neural information processing systems*. NeurIPS'21, Curran Associates, Inc., Virtual Only. https://openreview.net/forum?id=Wl32WBZnSP4.

Summers, C., & Dinneen, M.J. (2021). Nondeterminism and instability in neural network optimization. In *Proceedings of the 38th international conference on machine learning*. ICML'21. arXiv:2103.04514.

Sundararajan, M., Dhamdhere, K., Agarwal, A. (2020) The Shapley Taylor interaction index. In *Proceedings of the 37th international conference on machine learning*. ICML'20. http://proceedings.mlr.press/v119/sundararajan20a.

Sundararajan, M., & Najmi, A. (2020). The many Shapley values for model explanation. In *Proceedings of the 37th international conference on machine learning*. (pp. 9269–9278). ICML'20. arXiv:1908.08474

Tan, Y.C., Celis, L.E. (2019) Assessing social and intersectional biases in contextualized word representations. In *Proceedings of the 33rd conference on neural information processing systems*. NeurIPS'19, Curran Associates, Inc., arXiv:1911.01485

Terashita, N., Ohashi, H., Nonaka, Y., & Kanemaru, T. (2021). Influence estimation for generative adversarial networks. In *Proceedings of the 9th international conference on learning representations*. ICLR'21. arXiv:2101.08367.

Thimonier, H., Popineau, F., Rimmel, A., Doan, B.L., & Daniel, F. (2022). TracInAD: Measuring influence for anomaly detection. In: Proceedings of the 2022 international joint conference on neural networks. IJCNN'22. arXiv:2205.01362.

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society (Series B), 58*, 267–288.

Tietjen, G. L., Moore, R. H., & Beckman, R. J. (1973). Testing for a single outlier in simple linear regression. *Technometrics, 15*(4), 717–721.

Ting, D., & Brochu, E. (2018). Optimal subsampling with influence functions. In *Proceedings of the 32nd conference on neural information processing systems*. NeurIPS'18, Curran Associates, Inc. arXiv:1709.01716.

Tsai, C. P., Yeh, C. K., & Ravikumar, P. (2023). Faith-Shap: The faithful Shapley interaction index. *Journal of Machine Learning Research, 24*(94), 1–42.

Tsai, C.P., Zhang, J., Chien, E., Yu, H.F., Hsieh, C.J., & Ravikumar, P. (2023b). Representer point selection for explaining regularized high-dimensional models. In: Proceedings of the 40th international conference on machine learning. ICML'23. arXiv:2305.20002.

Tukan, M., Zhou, S., Maalouf, A., Rus, D., Braverman, V., & Feldman, D. (2023). Provable data subset selection for efficient neural networks training. In *Proceedings of the 40th international conference on machine learning*. ICML'23. arXiv:2303.05151

van den Burg, G.J.J., & Williams, C.K.I. (2021). On memorization in probabilistic deep generative models. In *Proceedings of the 35th conference on neural information processing systems*. NeurIPS'21, Curran Associates, Inc. arXiv:2106.03216

Wallace, E., Zhao, T.Z., Feng, S., & Singh, S. (2021). Concealed data poisoning attacks on NLP models. In *Proceedings of the North American chapter of the association for computational linguistics*. NAACL'21. arXiv:2010.12563.

Wang, B., Yao, Y., Shan, S., Li, H., Viswanath, B., Zheng, H., & Zhao, B.Y. (2019). Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *Proceedings of the 40th IEEE symposium on security and privacy*. SP'19. https://ieeexplore.ieee.org/document/8835365.

Wang, J.T., & Jia, R. (2023). Data Banzhaf: A robust data valuation framework for machine learning. In *Proceedings of the 26th international conference on artificial intelligence and statistics*. AISTATS'23. arXiv:2205.15466.

Wang, W., Levine, A., & Feizi, S. (2022). Improved certified defenses against data poisoning with (deterministic) finite aggregation. In *Proceedings of the 39th international conference on machine learning*. ICML'22. arXiv:2202.02628.

Wang, Z., Zhu, H., Dong, Z., He, X., & Huang, S. (2020). Less is better: Unweighted data subsampling via influence function. In *Proceedings of the 34th AAAI conference on artificial intelligence*. (pp. 6340–6347). AAAI'20, AAAI Press. arXiv:1912.01321

Wei, K., Iyer, R., Bilmes, J. (2015). Submodularity in data subset selection and active learning. In *Proceedings of the 32nd international conference on machine learning*. ICML'15, PMLR. https://proceedings.mlr.press/v37/wei15.html

Wojnowicz, M., Cruz, B., Zhao, X., Wallace, B., Wolff, M., Luan, J., & Crable, C. (2016). 'Influence sketching': Finding influential samples in large-scale regressions. In *Proceedings of the 2016 IEEE international conference on big data*. BigData'16, IEEE. arXiv:1611.05923.

Woodruff, D. P. (2014). Sketching as a tool for numerical linear algebra. *Foundations and Trends in Theoretical Computer Science, 10*(1–2), 1–157. arXiv:1411.4357.

Xiang, C. (2022). Scientists increasingly can't explain how AI works. Vice. https://www.vice.com/en/article/y3pezm/scientists-increasingly-cant-explain-how-ai-works

Yampolskiy, R. V. (2020). Unexplainability and incomprehensibility of AI. *Journal of Artificial Intelligence and Consciousness, 7*(2), 277–291.

Yan, T., & Procaccia, A.D. (2021). If you like Shapley then you'll love the core. In *Proceedings of the 35th AAAI conference on artificial intelligence*. AAAI'21, Association for the Advancement of Artificial Intelligence, Virtual Only. https://ojs.aaai.org/index.php/AAAI/article/view/16721.

Yang, J., Luo, L., Qian, J., Tai, Y., Zhang, F., & Xu, Y. (2017). Nuclear norm based matrix regression with applications to face recognition with occlusion and illumination changes. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 39*(1), 156–171. arXiv:1405.1207.

Yang, J., Zhou, K., Li, Y., & Liu, Z. (2021). *Generalized out-of-distribution detection: A survey. (vol. 2110, p. 11334*. arXiv:2110.11334.

Yang, S., Xie, Z., Peng, H., Xu, M., Sun, M., & Li, P. (2023) Dataset pruning: Reducing training data by examining generalization influence. In *Proceedings of the 11th international conference on learning representations*. ICLR'23. arXiv:2205.09329.

Yeh, C.K., Taly, A., Sundararajan, M., Liu, F., & Ravikumar, P. (2022). First is better than last for language data influence. In *Proceedings of the 36th conference on neural information processing systems*. NeurIPS'22, Curran Associates. arXiv:2202.11844

Yeh, C., Kim, J.S., Yen, I.E., & Ravikumar, P. (2018) Representer point selection for explaining deep neural networks. In *Proceedings of the 32nd conference on neural information processing systems*. NeurIPS'18, Curran Associates, Inc., Montreal. arXiv:1811.09720.

You, W., Hammoudeh, Z., & Lowd, D. (2023). Large language models are better adversaries: Exploring generative clean-label backdoor attacks against text classifiers. In *Findings of the association for computational linguistics*. EMNLP'23.

You, Y., Gitman, I., & Ginsburg, B. (2017). Large batch training of convolutional networks. arXiv:1708.03888.

Yuan, M., Ekici, A., Lu, Z., & Monteiro, R. (2007). Dimension reduction and coefficient estimation in multivariate linear regression. *Journal of the Royal Statistical Society: Series B (Statistical Methodology), 69*(3), 329–346.

Zeng, Y., Wang, J.T., Chen, S., Just, H.A., Jin, R., & Jia, R. (2023). ModelPred: A framework for predicting trained model from training data. In *Proceedings of the 1st IEEE conference on secure and trustworthy machine learning*. SaTML'23. arXiv:2111.12545

Zhang, C., Bengio, S., Hardt, M., Recht, B., & Vinyals, O. (2017). Understanding deep learning requires rethinking generalization. In *Proceedings of the 5th international conference on learning representations*. ICLR'17. arXiv:1611.03530

Zhang, C., Bengio, S., Hardt, M., Recht, B., & Vinyals, O. (2021). Understanding deep learning (still) requires rethinking generalization. *Communications of the ACM, 64*(3), 107–115. https://doi.org/10.1145/3446776

Zhang, C., Ippolito, D., Lee, K., Jagielski, M., Tramer F., & Carlini, N. (2021b). Counterfactual memorization in neural language models. arXiv:2112.12938.

Zhang, H., Lu, A.X., Abdalla, M., McDermott, M., & Ghassemi, M. (2020). Hurtful words: Quantifying biases in clinical contextual word embeddings. In *Proceedings of the ACM conference on health, inference, and learning*. CHIL'20, Association for Computing Machinery. arXiv:2003.11515.

Zhang, R., & Zhang, S. (2022). Rethinking influence functions of neural networks in the over-parameterized regime. In *Proceedings of the 36th AAAI conference on artificial intelligence*. AAAI'22, Association for the Advancement of Artificial Intelligence. arXiv:2112.08297.

Zhang, W., Wang, Y., You, Z., Cao, M., Huang, P., Shan, J., Yang, Z., Cui, B. (2021c). RIM: Reliable influence-based active learning on graphs. In *Proceedings of the 35th conference on neural information processing systems*. NeurIPS'21, Curran Associates, Inc., Virtual Only. arXiv:2110.14854.

Zhou, J., Li, Z., Hu, H., Yu, K., Chen, F., Li, Z., & Wang, Y. (2019) Effects of influence on user trust in predictive decision making. In *Extended Abstracts of the 2019 conference on human factors in computing systems*. CHI'19, Association for Computing Machinery.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.